



FEU INSTITUTE OF TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

**FINAL PROJECT
IN
CS0005
(Database Systems with Administration)**

**JVS Student Management Record System with PL/SQL
Implementation**

Group Name:	<i>Share Q Lang(SQL)</i>
Leader:	<i>Vince Carlo I. Lachica</i>
Members:	<i>Janelle P. Facto Rasheed L. Librorania</i>
Section:	<i>TN21</i>
Professor:	<i>Mr. Anthony D. Aquino</i>

PL/SQL Project Lab Manual

Title: Student Record Management System with PL/SQL Implementation

Course: CS0005 Database Systems with Administration

Tool: Oracle Database (Oracle APEX / Oracle SQL Developer)

Project Overview

This lab project focuses on developing a **System using PL/SQL**. Students will design tables, write PL/SQL blocks, create procedures, functions, triggers, cursors, and exception handling mechanisms.

Objectives

By the end of this lab, students should be able to:

1. Create and manage database tables with a minimum of 100 records.
 2. Write anonymous PL/SQL blocks.
 3. Develop stored procedures and functions.
 4. Use cursors (implicit and explicit).
 5. Implement triggers.
 6. Apply exception handling.
 7. Design a mini database-driven system using PL/SQL.
-

Project Scenario

A school wants to automate grade computation for students. The system should:

- Store student, professor, subject, and grades records.
 - Automate credential generation and have data validation
 - Compute midterm and final grades
 - Determine academic status automatically
 - Generate GWA (General Weighted Average)
 - Log and display student records
-

PART 1 – Database Setup

Task 1: Create Tables

Source Code:

```
-- tables.sql

BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE grades CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE subjects CASCADE CONSTRAINTS';
```

```
EXECUTE IMMEDIATE 'DROP TABLE students CASCADE CONSTRAINTS';
EXECUTE IMMEDIATE 'DROP TABLE teachers CASCADE CONSTRAINTS';
EXECUTE IMMEDIATE 'DROP TABLE audits CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE TABLE teachers (
    username    VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    password    VARCHAR2(20)
);

CREATE TABLE students (
    student_id  VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    section     VARCHAR2(10),
    password    VARCHAR2(20)
);

CREATE TABLE subjects (
    subject_code VARCHAR2(15) PRIMARY KEY,
    subject_name  VARCHAR2(100),
    professor     VARCHAR2(50)
);

CREATE TABLE grades (
    student_id    VARCHAR2(15),
    subject_code  VARCHAR2(15),
    sa NUMBER, ta NUMBER, sw NUMBER, rec NUMBER, fp NUMBER,
    me NUMBER, fe NUMBER,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES
students(student_id),
    CONSTRAINT fk_subject FOREIGN KEY (subject_code) REFERENCES
subjects(subject_code)
);

CREATE TABLE audits (
    log_id       NUMBER PRIMARY KEY,
    log_message  VARCHAR2(4000),
    log_date     DATE,
    db_user      VARCHAR2(100)
);
```

Code Analysis:

1. Table Cleanup (Drop Existing Tables)

```
BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE grades CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE subjects CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE students CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE teachers CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE audits CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

Purpose: This block removes existing tables before recreating them. This allows the script to run multiple times without errors.

Components:

- **SET FEEDBACK OFF** hides the message output of how many rows were affected by a command
- **EXECUTE IMMEDIATE** allows dynamic SQL execution.
- **CASCADE CONSTRAINTS** are used when dropping a table to ensure the table can be removed even if other tables have foreign key relationships referencing it.
- **WHEN OTHERS THEN NULL** prevents errors if the tables do not exist.

2. Create tables

```
CREATE TABLE teachers (
    username  VARCHAR2(15) PRIMARY KEY,
    name      VARCHAR2(100),
    password  VARCHAR2(20)
);

CREATE TABLE students (
    student_id VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    section     VARCHAR2(10),
    password    VARCHAR2(20)
);

CREATE TABLE subjects (
    subject_code VARCHAR2(15) PRIMARY KEY,
    subject_name  VARCHAR2(100),
```

```

        professor    VARCHAR2(50)
    );

CREATE TABLE grades (
    student_id    VARCHAR2(15),
    subject_code  VARCHAR2(15),
    sa NUMBER, ta NUMBER, sw NUMBER, rec NUMBER, fp NUMBER,
    me NUMBER, fe NUMBER,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES
students(student_id),
    CONSTRAINT fk_subject FOREIGN KEY (subject_code) REFERENCES
subjects(subject_code)
);

CREATE TABLE audits (
    log_id        NUMBER PRIMARY KEY,
    log_message   VARCHAR2(4000),
    log_date      DATE,
    db_user       VARCHAR2(100)
);

```

Purpose: This statement creates tables for students, teachers, subjects, grades, and audit log, that store data in the database.

Task 2: Insert 100 records

Source Code;

```

-- insert_records.sql

INSERT INTO subjects VALUES ('CS0021', 'DISCRETE STRUCTURES', 'Marilyn Sanchez');
INSERT INTO subjects VALUES ('CS0017', 'OPERATING SYSTEM', 'Jeneffer Sabonsolin');
INSERT INTO subjects VALUES ('CS0010', 'FUNDAMENTALS OF ANALYTICS', 'Anthony Aquino');
INSERT INTO subjects VALUES ('CS0005', 'DATABASE SYSTEMS', 'Fanny Almeniana');
INSERT INTO subjects VALUES ('CS0007', 'ALGORITHM', 'Justine Jude Pura');
INSERT INTO subjects VALUES ('CCS0101', 'DESIGN THINKING', 'May Florence San Pablo');

INSERT INTO teachers VALUES ('mmsanchez', 'Marilyn Sanchez', 'prof_mmsanchez');
INSERT INTO teachers VALUES ('jjsabonsolin', 'Jeneffer Sabonsolin', 'prof_jjsabonsolin');

```

```

INSERT INTO teachers VALUES ('aaaquino', 'Anthony Aquino', 'prof_aaaquino');
INSERT INTO teachers VALUES ('ffalmeniana', 'Fanny Almeniana',
'prof_ffalmeniana');
INSERT INTO teachers VALUES ('jjpura', 'Justine Jude Pura', 'prof_jjpura');
INSERT INTO teachers VALUES ('mmsanpablo', 'May Florence San Pablo',
'prof_mmsanpablo');

DECLARE
    TYPE t_name IS TABLE OF VARCHAR2(50);
    v_fn t_name := t_name(

'Aaron','Abigail','Adrian','Aileen','Albert','Alex','Alicia','Alvin','Amanda','An
drew',

'Angela','Angelo','Anita','Anthony','Arlene','Arnold','Arthur','Beatrice','Benjie
','Bernadette',

'Bethany','Bianca','Bobby','Brenda','Bryan','Camille','Carl','Carmen','Cedric','C
eline',

'Charles','Chloe','Chris','Clara','Claude','Cynthia','Daisy','Daniel','Dante','Da
phne',

'David','Debbie','Dennis','Diana','Diego','Dominic','Donna','Edgar','Edith','Edua
rdo',

'Edwin','Elaine','Eleanor','Elias','Elisa','Elizabeth','Ellen','Elmer','Elsa','Em
manuel',

'Eric','Erica','Esther','Eugene','Eunice','Eva','Evelyn','Felix','Ferdinand','Fer
nando',

'Fiona','Flora','Florence','Francis','Fred','Gabriel','Gemma','George','Gerald','
Gina',

'Gloria','Grace','Gregorio','Hannah','Harold','Hazel','Hector','Helen','Henry','H
erbert',

'Herman','Homer','Ida','Ignacio','Imelda','Irene','Iris','Isaac','Isabel','Isidro
',

    );

    v_ln t_name := t_name(

```

```

'Abad','Abalos','Abaya','Abril','Agoncillo','Aguinaldo','Alcasid','Almazan','Alme
da','Alonzo',

'Alvarado','Alvarez','Amante','Andrada','Angeles','Antipolo','Apostol','Aragon','
Araneta','Arceo',

'Arellano','Arnaiz','Arroyo','Asis','Atienza','Austria','Balagtas','Baluyot','Ban
ayo','Barretto',

'Basco','Belmonte','Benitez','Bernardo','Blanco','Bonifacio','Borja','Buenaventur
a','Buencamino','Burgos',

'Caballero','Cabanatuan','Cabrera','Cacho','Calaguas','Calderon','Calungsod','Cam
pos','Canlas','Caparas',

'Carreon','Castillo','Castro','Catacutan','Cayetano','Cervantes','Chu','Claudio',
'Co','Concepcion',

'Constantino','Coronel','Cortez','Crisostomo','Cruz','Cuenco','Cunanan','Custodio
','Dagohoy','Dandan',
    'David','Dayrit','De Guzman','De Jesus','De Leon','De Mesa','Del
Rosario','Delos Reyes','Delos Santos','Diamante',

'Diaz','Dilangalen','Dimaculangan','Dimalanta','Dimasalang','Diokno','Domingo','D
ominguez','Dorado','Dumlao',

'Echeverria','Ejercito','Encarnacion','Enrile','Enriquez','Escudero','Esguerra','
Espina','Espinosa','Espiritu'
    );

    v_sid VARCHAR2(15);
BEGIN
    FOR i IN 1..100 LOOP
        v_sid := '2024' || TRUNC(DBMS_RANDOM.VALUE(10000,99999));

        INSERT INTO students VALUES (
            v_sid,
            UPPER(v_fn(i) || ' ' || v_ln(i)),
            'TN2' || TRUNC(DBMS_RANDOM.VALUE(1,10)),
            v_sid
        );
    
```

```

FOR r_sub IN (SELECT subject_code FROM subjects) LOOP
    INSERT INTO grades VALUES (
        v_sid,
        r_sub.subject_code,
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98))
    );
END LOOP;
END LOOP;
COMMIT;
END;
/

PROMPT Records Inserted Successfully.

```

Output Screenshot:

Code Analysis:

1. Insert records into subjects table

```

INSERT INTO subjects VALUES ('CS0021', 'DISCRETE STRUCTURES', 'Marilyn Sanchez');
INSERT INTO subjects VALUES ('CS0017', 'OPERATING SYSTEM', 'Jeneffer
Sabonsolin');
INSERT INTO subjects VALUES ('CS0010', 'FUNDAMENTALS OF ANALYTICS', 'Anthony
Aquino');
INSERT INTO subjects VALUES ('CS0005', 'DATABASE SYSTEMS', 'Fanny Almeniana');
INSERT INTO subjects VALUES ('CS0007', 'ALGORITHM', 'Justine Jude Pura');
INSERT INTO subjects VALUES ('CCS0101', 'DESIGN THINKING', 'May Florence San
Pablo');

```

Purpose: These statements preload the database with 6 subjects and their assigned professors.

Components:

- **subject_code**
 - description: unique code for the subject
 - ex: CS0021

- **subject_name**
 - description: name of the subject/course
 - ex: DISCRETE STRUCTURES
- **Professor**
 - description: name of the professor teaching
 - ex: Marilyn Sanchez

2. Insert records into teachers table

```
INSERT INTO teachers VALUES ('mmsanchez', 'Marilyn Sanchez', 'prof_mmsanchez');
INSERT INTO teachers VALUES ('jjsabonsolin', 'Jeneffer Sabonsolin',
'prof_jjsabonsolin');
INSERT INTO teachers VALUES ('aaaquino', 'Anthony Aquino', 'prof_aaaquino');
INSERT INTO teachers VALUES ('ffalmeniana', 'Fanny Almeniana',
'prof_ffalmeniana');
INSERT INTO teachers VALUES ('jjpura', 'Justine Jude Pura', 'prof_jjpura');
INSERT INTO teachers VALUES ('mmsanpablo', 'May Florence San Pablo',
'prof_mmsanpablo');
```

Purpose: These statements insert teacher records into the teachers table. Each record stores the teacher's username, full name and password, This is mainly used with program access.

Components:

- **teacher_username**
 - description: unique username for the teacher
 - ex: mmsanpablo
- **teacher_name**
 - description: name of the teacher name
 - ex: May Florence San Pablo
- **teacher_password**
 - description: name of the password
 - ex: prof_mmsanpablo

3. Insert records into subjects table

```
DECLARE
    TYPE t_name IS TABLE OF VARCHAR2(50);
    v_fn t_name := t_name(
'Aaron','Abigail','Adrian','Aileen','Albert','Alex','Alicia','Alvin','Aman
da','Andrew',
```

```
'Angela','Angelo','Anita','Anthony','Arlene','Arnold','Arthur','Beatrice',  
'Benjie','Bernadette',  
  
'Bethany','Bianca','Bobby','Brenda','Bryan','Camille','Carl','Carmen','Ced  
ric','Celine',  
  
'Charles','Chloe','Chris','Clara','Claude','Cynthia','Daisy','Daniel','Dan  
te','Daphne',  
  
'David','Debbie','Dennis','Diana','Diego','Dominic','Donna','Edgar','Edith  
,','Eduardo',  
  
'Edwin','Elaine','Eleanor','Elias','Elisa','Elizabeth','Ellen','Elmer','El  
sa','Emmanuel',  
  
'Eric','Erica','Esther','Eugene','Eunice','Eva','Evelyn','Felix','Ferdinan  
d','Fernando',  
  
'Fiona','Flora','Florence','Francis','Fred','Gabriel','Gemma','George','Ge  
rald','Gina',  
  
'Gloria','Grace','Gregorio','Hannah','Harold','Hazel','Hector','Helen','He  
nry','Herbert',  
  
'Herman','Homer','Ida','Ignacio','Imelda','Irene','Iris','Isaac','Isabel',  
'Isidro'  
    );  
  
    v_ln t_name := t_name(  
  
'Abad','Abalos','Abaya','Abril','Agoncillo','Aguinaldo','Alcasid','Almazan  
,','Almeda','Alonzo',  
  
'Alvarado','Alvarez','Amante','Andrada','Angeles','Antipolo','Apostol','Ar  
agon','Araneta','Arceo',  
  
'Arellano','Arnaiz','Arroyo','Asis','Atienza','Austria','Balagtas','Baluyo  
t','Banayo','Barretto',
```

```

'Basco','Belmonte','Benitez','Bernardo','Blanco','Bonifacio','Borja','Buen
aventura','Buencamino','Burgos',

'Caballero','Cabanatuan','Cabrera','Cacho','Calaguas','Calderon','Calungso
d','Campos','Canlas','Caparas',

'Carreon','Castillo','Castro','Catacutan','Cayetano','Cervantes','Chu','Cl
audio','Co','Concepcion',

'Constantino','Coronel','Cortez','Crisostomo','Cruz','Cuenco','Cunanan','C
ustodio','Dagohoy','Dandan',
    'David','Dayrit','De Guzman','De Jesus','De Leon','De Mesa','Del
Rosario','Delos Reyes','Delos Santos','Diamante',

'Diaz','Dilangalen','Dimaculangan','Dimalanta','Dimasalang','Diokno','Domi
ngo','Dominguez','Dorado','Dumlao',

'Echeverria','Ejercito','Encarnacion','Enrile','Enriquez','Escudero','Esgu
erra','Espina','Espinosa','Espiritu'
    );

v_sid VARCHAR2(15);

```

Purpose: This block handles Dynamic Data Generation. Instead of manually encoding 100 students, we defined two arrays, v_fn for first names and v_ln for last names. The system uses these pools to automatically generate 100 unique student identities with randomized grades.

Components:

- **DECLARE**
TYPE t_name IS TABLE OF VARCHAR2(50);
 - this line creates a PL/SQL collection type called t_name that can store a list (array) of strings, where each string can be up to 50 characters.
- **v_fn t_name := t_name(**
'Aaron','Abigail','Adrian', ... , 'Isidro'
);
 - this block of code uses Array Handling and Randomization to create random first names
- **v_ln t_name := t_name(**
'Abad','Abalos','Abaya', ..., 'Espiritu'
);

- this block of code uses Array Handling and Randomization to create random last names
- **v_sid VARCHAR2(15);**
 - declares a variable to hold the student ID that can be up to 15 characters long. The system generates a random number that starts with 2024 and is 9 characters long, stores it in v_sid and inserts it into the students table.

4. Student ID, Section and Grades Generation

```
BEGIN
  FOR i IN 1..100 LOOP
    v_sid := '2024' || TRUNC(DBMS_RANDOM.VALUE(10000,99999));

    INSERT INTO students VALUES (
      v_sid,
      UPPER(v_fn(i) || ' ' || v_ln(i)),
      'TN2' || TRUNC(DBMS_RANDOM.VALUE(1,10)),
      v_sid
    );

    FOR r_sub IN (SELECT subject_code FROM subjects) LOOP
      INSERT INTO grades VALUES (
        v_sid,
        r_sub.subject_code,
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98))
      );
    END LOOP;
  END LOOP;
  COMMIT;
END;
/

PROMPT Records Inserted Successfully.
```

Purpose: This block is the Auto-Generator. It uses nested loops to automatically create 100 students and enroll them in all available subjects. String concatenation is used to build unique IDs

and randomization to generate their grades. This allows the system to process large amounts of data and compute if the students passed or failed.

Components:

- **FOR i IN 1..100 LOOP**
 v_sid := '2024' || TRUNC(DBMS_RANDOM.VALUE(10000,99999));
 - creates 100 unique random student numbers that starts with 2024 and consists of 9 digits
- **INSERT INTO students VALUES (**
 v_sid,
 UPPER(v_fn(i) || ' ' || v_ln(i)),
 'TN2' || TRUNC(DBMS_RANDOM.VALUE(1,10)),
 v_sid
);
 - takes the previously generated student ids, first names and last names, sections that starts with TN2 and ends with 1-9 randomly, and the password which is similar to the student number, and adds it to the students table.
- **FOR r_sub IN (SELECT subject_code FROM subjects) LOOP**
 INSERT INTO grades VALUES (
 v_sid,
 r_sub.subject_code,
 TRUNC(DBMS_RANDOM.VALUE(50,98)),
 TRUNC(DBMS_RANDOM.VALUE(50,98)),
 TRUNC(DBMS_RANDOM.VALUE(50,98)),
 TRUNC(DBMS_RANDOM.VALUE(50,98)),
 TRUNC(DBMS_RANDOM.VALUE(50,98)),
 TRUNC(DBMS_RANDOM.VALUE(50,98))
);
 END LOOP;
 - a nested loop to iterate through our subjects table for every student created. For each subject, the system generates randomized scores between 80 and 98 for all grading components. Finally, we use the COMMIT command to permanently save all these records to the database.
- **END LOOP;**
 COMMIT;
 END;
 /

PROMPT Success

- closes the loop earlier and saves the data permanently into the tables via the COMMIT command. The END and the / executes the entire PL/SQL block. The PROMPT command allows the user to see that the code has been run successfully.

PART 2 – Anonymous PL/SQL Block

Task 3: Create Anonymous PL/SQL Block

1. Midterm Grade Calculation

```
BEGIN
    v_avg := (p_sa + p_ta + p_sw + p_rec) / 4;
    RETURN ROUND((0.6 * v_avg) + (0.4 * p_me));
END;
/
```

Purpose: This calculates a student's midterm grade by first averaging the Summative Assessments (SA), Technical Assessments (TA), Seat Works (SW), and Recitations (REC). It then combines this average with the Midterm Exam (ME) using a formula and returns the rounded midterm grade.

2. Final Grade Calculation

```
BEGIN
    v_avg := (p_sa + p_ta + p_sw + p_rec + p_fp) / 5;
    RETURN ROUND((0.6 * v_avg) + (0.15 * p_me) + (0.25 * p_fe));
END;
/
```

Purpose: This calculates a student's final grade by first averaging the Summative Assessments (SA), Technical Assessments (TA), Seat Works (SW), Recitations (REC), and Final Project (FP). It then combines this average with the Midterm Exam (ME) and Final Exam (FE) and returns the rounded final grade.

3. General Weighted Average Calculation

```
CREATE OR REPLACE FUNCTION fn_calculate_gwa(p_mid IN NUMBER, p_fin IN NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN ROUND((p_mid + p_fin) / 2);
END;
```

/

Purpose: This calculates the General Weighted Average (GWA) by getting the rounded average of the student's midterm and final grades.

4. Student Status Evaluation

```
CREATE OR REPLACE FUNCTION fn_get_status(p_gwa IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
    IF p_gwa >= 70 THEN
        RETURN 'PASSED';
    ELSE
        RETURN 'FAILED';
    END IF;
END;
/
```

Purpose: This evaluates a student's General Weighted Average (GWA) and returns 'PASSED' if the GWA is 70 or higher, or 'FAILED' if it is below 70.

5. Student Login Authentication

```
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM students
    WHERE student_id = '&input_id' AND password = '&input_pass';

    :v_auth_count := v_count;
END;
/
```

Purpose: This verifies a student's login credentials by checking if the provided student ID and password exist in the students table. It stores the result in the variable :v_auth_count.

6. Database Table Cleanup

```
BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE grades CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE subjects CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE students CASCADE CONSTRAINTS';
```

```

EXECUTE IMMEDIATE 'DROP TABLE teachers CASCADE CONSTRAINTS';
EXECUTE IMMEDIATE 'DROP TABLE audits CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

```

Purpose: This block removes existing tables before recreating them. This allows the script to run multiple times without errors.

7. Student and Grade Data Generation

```

BEGIN
  FOR i IN 1..100 LOOP
    v_sid := '2024' || TRUNC(DBMS_RANDOM.VALUE(10000,99999));

    INSERT INTO students VALUES (
      v_sid,
      UPPER(v_fn(i) || ' ' || v_ln(i)),
      'TN2' || TRUNC(DBMS_RANDOM.VALUE(1,10)),
      v_sid
    );

    FOR r_sub IN (SELECT subject_code FROM subjects) LOOP
      INSERT INTO grades VALUES (
        v_sid,
        r_sub.subject_code,
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98)),
        TRUNC(DBMS_RANDOM.VALUE(50,98))
      );
    END LOOP;
  END LOOP;
  COMMIT;
END;
/

```


Purpose: This uses nested loops to automatically create 100 students and enroll them in all available subjects. String concatenation is used to build unique IDs and randomization to generate their grades. This allows the system to process large amounts of data and compute if the students passed or failed.

8. Display Student Grades

```
BEGIN
    SELECT COUNT(*) INTO v_total FROM grades WHERE student_id = v_id;

    OPEN c_item;
    FETCH c_item INTO r;

    IF c_item%FOUND THEN
        v_midterm := fn_calculate_midterm(r.sa, r.ta, r.sw, r.rec, r.me);
        v_final   := fn_calculate_final(r.sa, r.ta, r.sw, r.rec, r.fp, r.me,
r.fe);
        v_gwa      := fn_calculate_gwa(v_midterm, v_final);
        v_status   := fn_get_status(v_gwa);

        DBMS_OUTPUT.PUT_LINE('STUDENT NO : ' || v_id);
        DBMS_OUTPUT.PUT_LINE('NAME       : ' || UPPER(r.name));
        DBMS_OUTPUT.PUT_LINE('SECTION  : ' || r.section || ' | Subject ' ||
v_current || ' of ' || v_total);

        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('COURSE : ' || r.subject_code || ' - ' ||
r.subject_name);
        DBMS_OUTPUT.PUT_LINE('PROF   : ' || r.professor);

        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('=====');

        DBMS_OUTPUT.PUT_LINE(RPAD('SA',10)||RPAD('TA',10)||RPAD('SW',10)||RPAD('REC',10)|
|RPAD('FP',10)||RPAD('ME',10)||'FE');

        DBMS_OUTPUT.PUT_LINE(RPAD(r.sa,10)||RPAD(r.ta,10)||RPAD(r.sw,10)||RPAD(r.rec,10)|
|RPAD(r.fp,10)||RPAD(r.me,10)||r.fe);

        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('=====');
```

```

DBMS_OUTPUT.PUT_LINE(RPAD('MIDTERM',20)||RPAD('FINAL',20)||RPAD('GWA',20)||'STATUS');
        DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(v_midterm,
'990'),20)||RPAD(TO_CHAR(v_final, '990'),20)||RPAD(TO_CHAR(v_gwa,
'990'),20)||v_status);

DBMS_OUTPUT.PUT_LINE('=====');
=====');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No more subjects to display. ');
    END IF;
    CLOSE c_item;
END;
/

```

Purpose: This retrieves grades from the grades database and displays the active student's grades. It calculates the grades using the user-defined functions and formats the table displayed. It also displays error handling when no records are found.

9. Teacher Login Verification

```

BEGIN
    SELECT COUNT(*) INTO v_count
    FROM teachers
    WHERE username = '&input_id' AND password = '&input_pass';

    :v_auth_count := v_count;
END;
/

```

Purpose: This verifies a teacher's login credentials by checking if the provided username and password exist in the teachers table. It counts the matching records and stores the result in the variable :v_auth_count

10. Add Student Record Logic

```

BEGIN
    add_student('&v_sid');
END;
/

```

Purpose: Calls the stored procedure for adding students.

11. Add Professor Record Logic

```
BEGIN
    add_professor('&v_full_name', '&v_scode');
END;
/
```

Purpose: Calls the stored procedure for adding professors.

12. Add Subject Record Logic

```
BEGIN
    add_subject('&v_scode', '&v_sname');
END;
/
```

Purpose: Calls the stored procedure for adding subjects.

13. Deletion Logic

```
COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE '&v_del_choice'
        WHEN '1' THEN 'delete_students.sql'
        WHEN '2' THEN 'delete_teachers.sql'
        WHEN '3' THEN 'delete_subjects.sql'
        WHEN '4' THEN 'teacher_menu.sql'
        ELSE 'teacher_delete_records.sql'
    END AS next_scr
FROM dual;

BEGIN
    SELECT COUNT(*) INTO v_check FROM students WHERE student_id = '&v_id';
    IF v_check > 0 THEN
        DELETE FROM grades WHERE student_id = '&v_id';
        DELETE FROM students WHERE student_id = '&v_id';

        COMMIT;
        DBMS_OUTPUT.PUT_LINE('&v_id successfully deleted from the table.');
```

```
ELSE
```

```

        DBMS_OUTPUT.PUT_LINE('No record found for ID &v_id.');
```

END IF;

EXCEPTION

 WHEN OTHERS THEN

 ROLLBACK;

 DBMS_OUTPUT.PUT_LINE('[Error] ' || SQLERRM);

END;

/

BEGIN

 SELECT COUNT(*) INTO v_count FROM teachers WHERE username = '&v_user';

 IF v_count > 0 THEN

 SELECT name INTO v_name_found FROM teachers WHERE username = '&v_user';

 UPDATE subjects SET professor = 'TBA' WHERE professor = v_name_found;

 DELETE FROM teachers WHERE username = '&v_user';

 COMMIT;

 DBMS_OUTPUT.PUT_LINE('Professor ' || v_name_found || ' removed.');

 ELSE

 DBMS_OUTPUT.PUT_LINE('[Error] Professor Username "&v_user" not found.');

 END IF;

END;

/

BEGIN

 SELECT COUNT(*) INTO v_count FROM subjects WHERE subject_code = v_code;

 IF v_count > 0 THEN

 SELECT subject_name INTO v_name_found FROM subjects WHERE subject_code =

v_code;

 DELETE FROM grades WHERE subject_code = v_code;

 DELETE FROM subjects WHERE subject_code = v_code;

 COMMIT;

 DBMS_OUTPUT.PUT_LINE('Subject ' || v_code || ' - ' || v_name_found || ' removed.');

 ELSE

 DBMS_OUTPUT.PUT_LINE('[Error] Subject Code ' || v_code || ' not found.');

 END IF;

END;

/

Purpose: This is a part of the Delete Records which allows the user to remove a record in the database. The process depends on whether the user chooses to remove a student, professor or subject record. If the user chooses to remove a student, it prompts the user to enter a student ID, find it in the database, and remove it. This is also the same process for the professor, and subject record. This also includes an error handling if the user inputs aren't found in the database or are invalid.

PART 3 – Stored Procedure

Task 4: Create Procedure

1. Update All Student Grades

```
CREATE OR REPLACE PROCEDURE update_all_grades AS
BEGIN
    UPDATE grades
    SET me = fn_calculate_midterm(sa, ta, sw, rec, me);

    UPDATE grades
    SET fe = fn_calculate_final(sa, ta, sw, rec, fp, me, fe);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('All Grades have been updated successfully.');
```

END;

Purpose: This updates all the grades for all students in the grades table. It uses the user-defined functions `fn_calculate_midterm` and `fn_calculate_finals` to compute the grades based on each student's assessments and existing scores. After updating all records, it commits the changes to the database and displays a confirmation message indicating that all grades have been successfully updated.

2. Add Student Record

```
CREATE OR REPLACE PROCEDURE add_student (
    p_sid IN VARCHAR2,
    p_sec IN VARCHAR2
) AS
BEGIN
    INSERT INTO students (student_id, name, section)
```

```
VALUES (TRIM(p_sid), 'NEW STUDENT', UPPER(TRIM(p_sec)));
END;
/
```

Purpose: This procedure inserts a new student record into the students table. It takes the student ID and section as input, trims any extra spaces, converts the section to uppercase, and assigns a default name 'NEW STUDENT'.

3. Add Teacher Record

```
CREATE OR REPLACE PROCEDURE add_teacher(
    p_full_name IN VARCHAR2,
    p_scode     IN VARCHAR2
) AS
    v_first_init   VARCHAR2(2);
    v_last_name    VARCHAR2(50);
    v_generated_user VARCHAR2(20);
    v_generated_pass VARCHAR2(30);
    v_check_subj   NUMBER;
BEGIN
    v_first_init := LOWER(SUBSTR(p_full_name, 1, 1)) || LOWER(SUBSTR(p_full_name,
1, 1));
    v_last_name  := LOWER(SUBSTR(p_full_name, INSTR(p_full_name, ' ', -1) + 1));
    v_generated_user := v_first_init || v_last_name;
    v_generated_pass := 'prof_' || v_generated_user;

    SELECT COUNT(*) INTO v_check_subj FROM subjects WHERE subject_code =
UPPER(p_scode);

    IF v_check_subj = 0 THEN
        DBMS_OUTPUT.PUT_LINE('[Error] Subject Code ' || p_scode || ' is invalid. ');
    ELSE
        INSERT INTO teachers (username, name, password)
        VALUES (v_generated_user, p_full_name, v_generated_pass);

        UPDATE subjects
        SET professor = p_full_name
        WHERE subject_code = UPPER(p_scode);

        COMMIT;
```

```

DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Professor Added Successfully. ');
    DBMS_OUTPUT.PUT_LINE('Username : ' || v_generated_user);
    DBMS_OUTPUT.PUT_LINE('Password : ' || v_generated_pass);
    DBMS_OUTPUT.PUT_LINE('Assigned Subject : ' || UPPER(p_scode));

DBMS_OUTPUT.PUT_LINE('=====');
END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[Error] Registration failed. Check name format or
duplicate user. ');
        ROLLBACK;
END;
/

```

Purpose: This procedure manages the logic for adding teachers to the system. It automatically generates a username and password for the teacher based on the provided full name and verifies if the given subject code exists in the subjects table. If the subject code is valid, it inserts the teacher's record into the teachers table and assigns the teacher to the subject by updating the subject record. It then displays the credentials along with a success message. If the subject code is invalid or an error occurs, error messages are shown.

8. Add Subject Record

```

CREATE OR REPLACE PROCEDURE add_subject(
    p_scode IN VARCHAR2,
    p_sname IN VARCHAR2
) AS
    v_check NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM subjects WHERE subject_code = UPPER(p_scode);

    IF v_check > 0 THEN
        DBMS_OUTPUT.PUT_LINE('[Error] Subject Code [' || UPPER(p_scode) || ']
already exists. ');
    ELSE
        INSERT INTO subjects (subject_code, subject_name, professor)
        VALUES (UPPER(p_scode), UPPER(p_sname), 'TBA');

        INSERT INTO grades (student_id, subject_code, sa, ta, sw, rec, fp, me, fe)
        SELECT student_id, UPPER(p_scode), 0, 0, 0, 0, 0, 0, 0 FROM students;
    END IF;
END;

```

```

        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Subject ' || UPPER(p_sname) || ' (' || UPPER(p_scode)
|| ') Added Successfully.');
```

```

    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[Error] Subject not added successfully.');
```

```

    ROLLBACK;
END;
/
```

Purpose: This procedure manages the logic for adding a new subject to the system. It first checks whether the subject code already exists in the subjects table to prevent duplicate entries. If the subject code is unique, it inserts the new subject into the subjects table with a default teacher assignment of TBA. It then automatically creates corresponding records in the grades table for all existing students, initializing all grade components to zero. The procedure commits the transaction upon success and displays a confirmation message. If a duplicate is detected or an error occurs, appropriate error messages are shown and the transaction is rolled back.

PART 4 – Function

Task 5: Create Function

1. Calculation for Midterm Grade

```

CREATE OR REPLACE FUNCTION fn_calculate_midterm(
    p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_me NUMBER
) RETURN NUMBER IS
    v_avg NUMBER;
BEGIN
    v_avg := (p_sa + p_ta + p_sw + p_rec) / 4;
    RETURN ROUND((0.6 * v_avg) + (0.4 * p_me));
END;
/
```

Purpose: This function automates the calculation of the Midterm Grade by taking five raw scores as inputs. It first calculates the average of four class-standing components, which accounts for 60% of the grade, and then adds the Midterm Exam score, which accounts for the remaining 40%. The final result is rounded to the nearest whole number.

2. Calculation for Final Grade

```
CREATE OR REPLACE FUNCTION fn_calculate_final(  
    p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_fp NUMBER, p_me  
    NUMBER, p_fe NUMBER  
) RETURN NUMBER IS  
    v_avg NUMBER;  
BEGIN  
    v_avg := (p_sa + p_ta + p_sw + p_rec + p_fp) / 5;  
    RETURN ROUND((0.6 * v_avg) + (0.15 * p_me) + (0.25 * p_fe));  
END;  
/
```

Purpose: This function automates the calculation of the Final Grade by taking seven raw scores as inputs. It first calculates the average of five class-standing components, which accounts for 60% of the grade, adds the Midterm Exam score, which accounts for 15% of the grade, and then adds the Final Exam score, which accounts for the remaining 25%. The final result is rounded to the nearest whole number.

3. Calculation for General Weighted Average (GWA)

```
CREATE OR REPLACE FUNCTION fn_calculate_gwa(p_mid IN NUMBER, p_fin IN NUMBER)  
RETURN NUMBER IS  
BEGIN  
    RETURN ROUND((p_mid + p_fin) / 2);  
END;  
/
```

Purpose: Takes the midterm grade and finals grade, adds both, divides the sum by two, and rounds the result to the nearest whole number.

4. Evaluates if the student passed or failed based on the General Weighted Average (GWA)

```
CREATE OR REPLACE FUNCTION fn_get_status(p_gwa IN NUMBER)  
RETURN VARCHAR2 IS  
BEGIN  
    IF p_gwa >= 70 THEN  
        RETURN 'PASSED';  
    ELSE  
        RETURN 'FAILED';  
    END IF;  
END;
```

```
END IF;  
END;  
/
```

Purpose: Uses a conditional statement to evaluate the student's GWA. If the GWA meets or exceeds 70, it returns the string 'PASSED', otherwise, it returns 'FAILED'.

PART 5 – Cursor

Task 6: Display All Records using Cursor

1. Usage of Explicit Cursor For Loop for Viewing Student Grades from Student's Perspective

```
DECLARE  
    v_id          VARCHAR2(15) := '&g_id';  
    v_current      NUMBER := TO_NUMBER('&v_pos');  
    v_total        NUMBER;  
  
    v_midterm      NUMBER;  
    v_final        NUMBER;  
    v_gwa          NUMBER;  
    v_status       VARCHAR2(15);  
  
    CURSOR c_item IS  
        SELECT * FROM (  
            SELECT s.name, s.section, sub.subject_code, sub.subject_name,  
            sub.professor,  
                g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe,  
                ROW_NUMBER() OVER (ORDER BY sub.subject_code) as rn  
            FROM students s  
            JOIN grades g ON s.student_id = g.student_id  
            JOIN subjects sub ON g.subject_code = sub.subject_code  
            WHERE s.student_id = v_id  
        ) WHERE rn = v_current;  
  
    r c_item%ROWTYPE;  
BEGIN  
    SELECT COUNT(*) INTO v_total FROM grades WHERE student_id = v_id;  
  
    OPEN c_item;
```

```

    FETCH c_item INTO r;

    IF c_item%FOUND THEN
        v_midterm := fn_calculate_midterm(r.sa, r.ta, r.sw, r.rec, r.me);
        v_final   := fn_calculate_final(r.sa, r.ta, r.sw, r.rec, r.fp, r.me,
r.fe);
        v_gwa     := fn_calculate_gwa(v_midterm, v_final);
        v_status  := fn_get_status(v_gwa);

        DBMS_OUTPUT.PUT_LINE('STUDENT NO : ' || v_id);
        DBMS_OUTPUT.PUT_LINE('NAME       : ' || UPPER(r.name));
        DBMS_OUTPUT.PUT_LINE('SECTION   : ' || r.section || ' | Subject ' ||
v_current || ' of ' || v_total);

        DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('COURSE : ' || r.subject_code || ' - ' ||
r.subject_name);
        DBMS_OUTPUT.PUT_LINE('PROF    : ' || r.professor);

        DBMS_OUTPUT.PUT_LINE('=====');

        DBMS_OUTPUT.PUT_LINE(RPAD('SA',10)||RPAD('TA',10)||RPAD('SW',10)||RPAD('REC',10)|
|RPAD('FP',10)||RPAD('ME',10)||'FE');

        DBMS_OUTPUT.PUT_LINE(RPAD(r.sa,10)||RPAD(r.ta,10)||RPAD(r.sw,10)||RPAD(r.rec,10)|
|RPAD(r.fp,10)||RPAD(r.me,10)||r.fe);

        DBMS_OUTPUT.PUT_LINE('=====');

        DBMS_OUTPUT.PUT_LINE(RPAD('MIDTERM',20)||RPAD('FINAL',20)||RPAD('GWA',20)||'STATU
S');
        DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(v_midterm,
'990'),20)||RPAD(TO_CHAR(v_final, '990'),20)||RPAD(TO_CHAR(v_gwa,
'990'),20)||v_status);

        DBMS_OUTPUT.PUT_LINE('=====');

    ELSE
        DBMS_OUTPUT.PUT_LINE('No more subjects to display.');
```

```
END IF;
```

```
CLOSE c_item;
```

```
END;  
/
```

Components:

- **Declare Block**

```
DECLARE  
    v_id          VARCHAR2(15) := '&g_id';  
    v_current      NUMBER := TO_NUMBER('&v_pos');  
    v_total        NUMBER;  
  
    v_midterm      NUMBER;  
    v_final        NUMBER;  
    v_gwa          NUMBER;  
    v_status       VARCHAR2(15);
```

- Stores inputs, intermediate values, and defines the data structure and query needed to retrieve and compute a student's grade details.

- **Cursor Declaration (c_grades)**

```
CURSOR c_item IS  
    SELECT * FROM (  
        SELECT s.name, s.section, sub.subject_code, sub.subject_name,  
        sub.professor,  
            g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe,  
            ROW_NUMBER() OVER (ORDER BY sub.subject_code) as rn  
        FROM students s  
        JOIN grades g ON s.student_id = g.student_id  
        JOIN subjects sub ON g.subject_code = sub.subject_code  
        WHERE s.student_id = v_id  
    ) WHERE rn = v_current;  
  
r c_item%ROWTYPE;
```

- Retrieves grade records for a specific student via usage of v_id by joining the students, grades, and subjects tables, and fetching the needed details.

- **BEGIN Block**

```
BEGIN  
  
    SELECT COUNT(*) INTO v_total FROM grades WHERE student_id = v_id;  
  
    OPEN c_item;  
    FETCH c_item INTO r;  
  
    IF c_item%FOUND THEN  
        v_midterm := fn_calculate_midterm(r.sa, r.ta, r.sw, r.rec,  
r.me);
```

```

        v_final      := fn_calculate_final(r.sa, r.ta, r.sw, r.rec, r.fp,
r.me, r.fe);
        v_gwa        := fn_calculate_gwa(v_midterm, v_final);
        v_status      := fn_get_status(v_gwa);

        DBMS_OUTPUT.PUT_LINE('STUDENT NO : ' || v_id);
        DBMS_OUTPUT.PUT_LINE('NAME          : ' || UPPER(r.name));
        DBMS_OUTPUT.PUT_LINE('SECTION      : ' || r.section || ' |
Subject ' || v_current || ' of ' || v_total);

DBMS_OUTPUT.PUT_LINE('=====
=====');
        DBMS_OUTPUT.PUT_LINE('COURSE : ' || r.subject_code || ' - '
|| r.subject_name);
        DBMS_OUTPUT.PUT_LINE('PROF      : ' || r.professor);

DBMS_OUTPUT.PUT_LINE('=====
=====');

DBMS_OUTPUT.PUT_LINE(RPAD('SA',10)||RPAD('TA',10)||RPAD('SW',10)||RP
AD('REC',10)||RPAD('FP',10)||RPAD('ME',10)||'FE');

DBMS_OUTPUT.PUT_LINE(RPAD(r.sa,10)||RPAD(r.ta,10)||RPAD(r.sw,10)||RP
AD(r.rec,10)||RPAD(r.fp,10)||RPAD(r.me,10)||r.fe);

DBMS_OUTPUT.PUT_LINE('=====
=====');

DBMS_OUTPUT.PUT_LINE(RPAD('MIDTERM',20)||RPAD('FINAL',20)||RPAD('GWA
',20)||'STATUS');
        DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(v_midterm,
'990'),20)||RPAD(TO_CHAR(v_final,      '990'),20)||RPAD(TO_CHAR(v_gwa,
'990'),20)||v_status);

DBMS_OUTPUT.PUT_LINE('=====
=====');

        ELSE
            DBMS_OUTPUT.PUT_LINE('No more subjects to display. ');
        END IF;
    CLOSE c_item;

```

```
END;  
/
```

- Initializes the process by preparing required data such as total subjects and retrieving the selected subject record. Validates retrieved data and computes the student's midterm, final grade, overall GWA, and academic status. Formats and displays the student's subject grades and computed results in a structured and readable output.

- **Navigation Block**

```
PROMPT [1] Next Subject  
PROMPT [2] Previous Subject  
PROMPT [3] Go back to Student Menu  
  
ACCEPT v_nav PROMPT 'What would you like to do next? '  
  
SET TERMOUT OFF  
COLUMN v_pos NEW_VALUE v_pos  
COLUMN next_script NEW_VALUE v_script  
  
SELECT  
    CASE '&v_nav'  
        WHEN '1' THEN TO_CHAR(&v_pos + 1)  
        WHEN '2' THEN TO_CHAR(CASE WHEN &v_pos > 1 THEN &v_pos - 1 ELSE 1 END)  
        ELSE TO_CHAR(&v_pos)  
    END AS v_pos,  
    CASE '&v_nav'  
        WHEN '3' THEN 'student_menu.sql'  
        ELSE 'student_view_grades.sql'  
    END AS next_script  
FROM dual;  
  
UNDEFINE v_nav  
SET TERMOUT ON  
  
@&v_script
```

- Controls user navigation between subjects by updating the current position and dynamically redirecting to the appropriate script based on the selected option.

2. Usage of Explicit Cursor For Loop for Viewing Student Grades from Teacher's Perspective

```
DECLARE
```

```

CURSOR c_filtered_students IS
    SELECT
        s.student_id,
        s.name,
        s.section,
        NVL(ROUND(AVG(fn_calculate_gwa(
            fn_calculate_midterm(g.sa, g.ta, g.sw, g.rec, g.me),
            fn_calculate_final(g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe)
        )), 0) as overall_gwa
    FROM students s
    LEFT JOIN grades g ON s.student_id = g.student_id
    WHERE UPPER(TRIM(s.section)) = UPPER(TRIM('&v_section'))
    GROUP BY s.student_id, s.name, s.section
    ORDER BY s.name ASC;

v_found  BOOLEAN := FALSE;
v_choice NUMBER := TO_NUMBER('&v_filter');
v_status VARCHAR2(10);
BEGIN
    IF v_choice = 4 THEN
        GOTO end_script;
    END IF;

    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(RPAD('ID', 15) || RPAD('NAME', 30) || RPAD('GWA', 10) ||
'STATUS');

    DBMS_OUTPUT.PUT_LINE('=====
=====');

    FOR r IN c_filtered_students LOOP

        IF r.overall_gwa >= 70 THEN
            v_status := 'PASSED';
        ELSE
            v_status := 'FAILED';
        END IF;

        IF (v_choice = 1) OR
            (v_choice = 2 AND v_status = 'PASSED') OR
            (v_choice = 3 AND v_status = 'FAILED') THEN

            DBMS_OUTPUT.PUT_LINE(

```

```

        RPAD(r.student_id, 15) ||
        RPAD(UPPER(r.name), 30) ||
        RPAD(TO_CHAR(r.overall_gwa), 10) ||
        v_status
    );
    v_found := TRUE;
END IF;
END LOOP;

IF NOT v_found THEN
    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('No records found for this filter/section.');
```

=====

```

END IF;

DBMS_OUTPUT.PUT_LINE('=====');

<<end_script>>
NULL;
END;
/
```

Components:

- **Cursor Declaration (c_filtered_students)**

```

CURSOR c_filtered_students IS
    SELECT
        s.student_id,
        s.name,
        s.section,
        NVL(ROUND(AVG(fn_calculate_gwa(
            fn_calculate_midterm(g.sa, g.ta, g.sw, g.rec, g.me),
            fn_calculate_final(g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe)
        )), 0) as overall_gwa
    FROM students s
    LEFT JOIN grades g ON s.student_id = g.student_id
    WHERE UPPER(TRIM(s.section)) = UPPER(TRIM('&v_section'))
    GROUP BY s.student_id, s.name, s.section
    ORDER BY s.name ASC;
```


- Retrieve the list of students for the given section along with a computed overall_gwa (safe default 0) for each student to be used in the report/filtering.

● Variable Declarations

```
v_found  BOOLEAN := FALSE;
v_choice NUMBER := TO_NUMBER('&v_filter');
v_status VARCHAR2(10);
```

- Prepare control variables used for filtering logic, output control, and storing per-row status.

● Filtering and Display

```
BEGIN
  IF v_choice = 4 THEN
    GOTO end_script;
  END IF;

  DBMS_OUTPUT.PUT_LINE(CHR(10));
  DBMS_OUTPUT.PUT_LINE(RPAD('ID', 15) || RPAD('NAME', 30) || RPAD('GWA', 10) ||
'STATUS');

  DBMS_OUTPUT.PUT_LINE('=====
=====');

  FOR r IN c_filtered_students LOOP

    IF r.overall_gwa >= 70 THEN
      v_status := 'PASSED';
    ELSE
      v_status := 'FAILED';
    END IF;

    IF (v_choice = 1) OR
      (v_choice = 2 AND v_status = 'PASSED') OR
      (v_choice = 3 AND v_status = 'FAILED') THEN

      DBMS_OUTPUT.PUT_LINE(
        RPAD(r.student_id, 15) ||
        RPAD(UPPER(r.name), 30) ||
        RPAD(TO_CHAR(r.overall_gwa), 10) ||
        v_status
      );
      v_found := TRUE;
    END IF;
  END LOOP;
```

```

END LOOP;

IF NOT v_found THEN
    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('No records found for this filter/section.');
```

```

END IF;

DBMS_OUTPUT.PUT_LINE('=====
=====');

<<end_script>>
NULL;
END;
/

```

- Filters and displays students from a selected section based on user-defined criteria, computes their overall GWA and status, and presents the results in a formatted list with handling empty outputs or early exit.

● Conditional Navigation

```

PAUSE Go back to previous menu?

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE
        WHEN '&v_filter' = '4' THEN 'teacher_view_records.sql'
        ELSE 'teacher_view_records_students.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@&v_next

```

- Determines which SQL script to run next based on a filter (v_filter) and executes it, effectively pausing to choose the appropriate teacher's view before returning to the previous menu.

3. Check if student record exists

```

SELECT COUNT(*) INTO v_count FROM students WHERE student_id = p_sid;

```

Purpose: Checks if a student exists in the database.

4. Retrieve Student Details

```
SELECT name, section INTO v_name, v_sec FROM students WHERE student_id = p_sid;
```

Purpose: Fetches the student's name and section for processing.

5. Select Student Subjects

```
SELECT p_sid, subject_code, 0, 0, 0, 0, 0, 0, 0
FROM subjects s
WHERE NOT EXISTS (
    SELECT 1 FROM grades g
    WHERE g.student_id = p_sid
    AND g.subject_code = s.subject_code
);
```

Purpose: Selects all subjects the student is not yet enrolled in and generates placeholder grades.

6. Check if subject record exists

```
SELECT COUNT(*) INTO v_check_subj FROM subjects WHERE subject_code = UPPER(p_scode);
```

Purpose: Checks if a subject exists in the database.

7. Student Login Authentication

```
SELECT student_id, UPPER(p_scode), 0, 0, 0, 0, 0, 0, 0 FROM students;
```

Purpose: Generates placeholder grade entries for all existing students for a newly added subject.

8. Student Login Authentication

```
SELECT COUNT(*) INTO v_count
FROM students
WHERE student_id = '&input_id' AND password = '&input_pass';
```

Purpose: Checks whether a student's login credentials are valid.

9. Retrieve Student ID

```
SELECT '&input_id' AS input_id FROM dual;
```

Purpose: Returns the input student ID.

10. Teacher Login Authentication

```
SELECT COUNT(*) INTO v_count  
FROM teachers  
WHERE username = '&input_id' AND password = '&input_pass';
```

Purpose: Checks whether a teacher's login credentials are valid.

11. Check if student exists for Deletion

```
SELECT COUNT(*) INTO v_count FROM students WHERE student_id = v_id_to_del;
```

Purpose: Determines if the student exists before deletion.

12. Retrieve Student name for Deletion

```
SELECT name INTO v_name_found FROM students WHERE student_id =  
v_id_to_del;
```

Purpose: Fetches the student's name before deletion.

13. Check if teacher exists for Deletion

```
SELECT COUNT(*) INTO v_count FROM teachers WHERE username = v_id_to_del;
```

Purpose: Determines if the teacher exists before deletion.

14. Retrieve teacher name for Deletion

```
SELECT name INTO v_name_found FROM teachers WHERE username = v_id_to_del;
```

Purpose: Determines if the teacher exists before deletion.

15. Check if subject exists for Deletion

```
SELECT COUNT(*) INTO v_count FROM subjects WHERE subject_code =  
v_id_to_del;
```

Purpose: Determines if the subject exists before deletion.

16. Retrieve subject name for Deletion

```
SELECT subject_name INTO v_name_found FROM subjects WHERE subject_code =  
v_id_to_del;
```

Purpose: Determines if the subject exists before deletion.

PART 6 – Trigger

Task 7: Create Trigger to Log Changes in Table

PART 7 – Exception Handling

Task 8: Handle Employee Not Found Error

1. Trigger for checking grade ranges

```
CREATE OR REPLACE TRIGGER trg_check_grade_range  
BEFORE INSERT OR UPDATE ON grades  
FOR EACH ROW  
BEGIN  
    IF :NEW.sa > 100 OR :NEW.ta > 100 OR :NEW.sw > 100 OR  
       :NEW.rec > 100 OR :NEW.me > 100 OR :NEW.fe > 100 THEN  
        RAISE_APPLICATION_ERROR(-20001, '[Trigger] Grades cannot be higher than  
100.');
```

Purpose: Makes sure grades don't go over 100 before saving them.

2. Trigger for calculating results

```
CREATE OR REPLACE TRIGGER trg_auto_calc_results
```

```

BEFORE UPDATE ON grades
FOR EACH ROW
DECLARE
    v_mid NUMBER;
    v_fin NUMBER;
BEGIN
    v_mid := fn_calculate_midterm(:NEW.sa, :NEW.ta, :NEW.sw, :NEW.rec, :NEW.me);
    v_fin :=
fn_calculate_final(:NEW.sa, :NEW.ta, :NEW.sw, :NEW.rec, :NEW.fp, :NEW.me, :NEW.fe
);

    DBMS_OUTPUT.PUT_LINE('[Trigger] Recalculated results for Student. '
|| :NEW.student_id);
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002, '[Trigger] Error. Check if function
parameters match.');
```

Purpose: Automatically recalculates a student's midterm and final when their grades are updated.

3. Log student deletion

```

CREATE OR REPLACE TRIGGER trg_audit_student_delete
AFTER DELETE ON students
FOR EACH ROW
BEGIN
    INSERT INTO audits (log_message, log_date, db_user)
    VALUES ('[Trigger] Deleted Student ID: ' || :OLD.student_id, SYSDATE, USER);
END;
/
```

Purpose: Logs an action whenever a student is deleted from the students table.

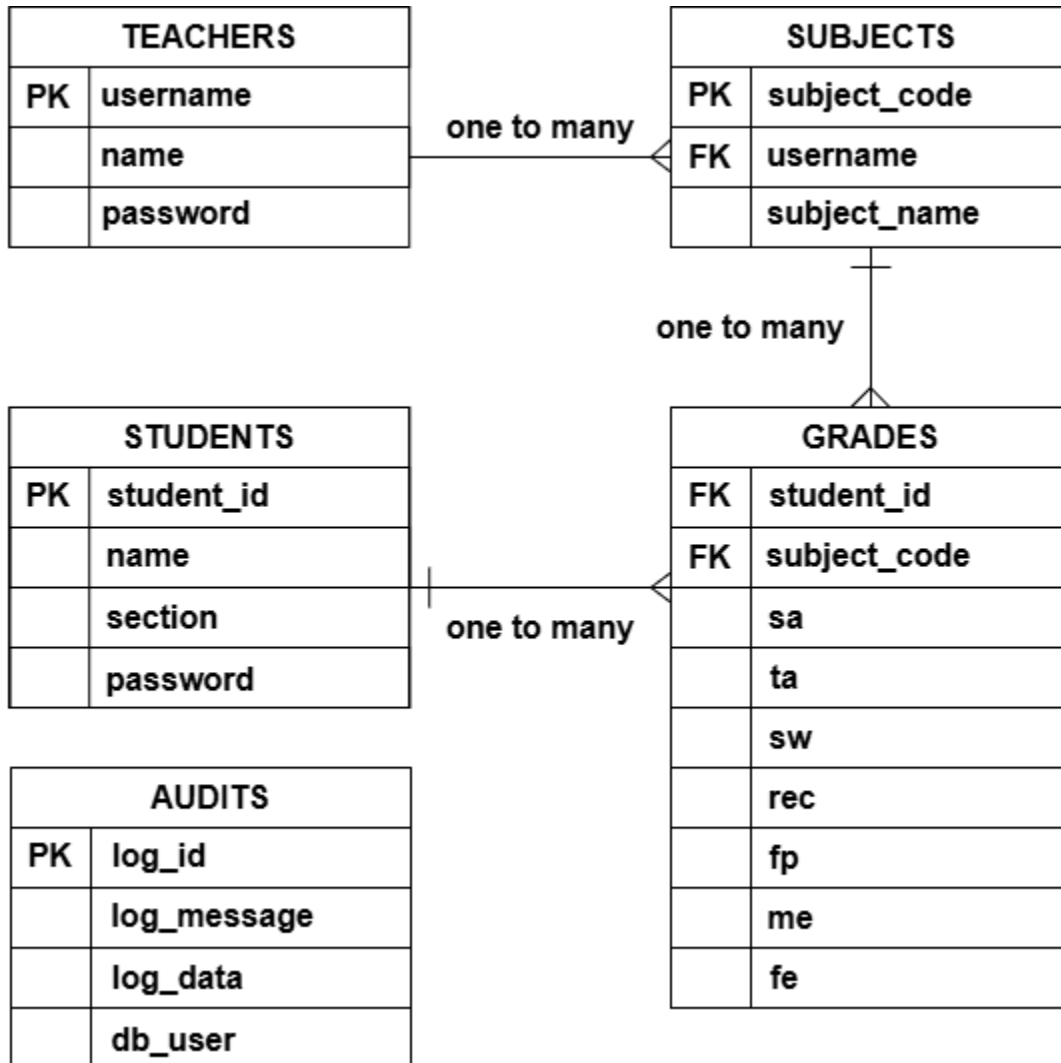
Deliverables:

1. Title Page
2. Objectives
3. ER Diagram
4. Table Structures
5. Source Code
6. Screenshots of Execution
7. Output Results

8. Conclusion

Deliverables:

3. ER Diagram



4. Table Structures

Source Code:

```
-- tables.sql

BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE grades CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE subjects CASCADE CONSTRAINTS';
```

```
EXECUTE IMMEDIATE 'DROP TABLE students CASCADE CONSTRAINTS';
EXECUTE IMMEDIATE 'DROP TABLE teachers CASCADE CONSTRAINTS';
EXECUTE IMMEDIATE 'DROP TABLE audits CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE TABLE teachers (
    username    VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    password    VARCHAR2(20)
);

CREATE TABLE students (
    student_id  VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    section     VARCHAR2(10),
    password    VARCHAR2(20)
);

CREATE TABLE subjects (
    subject_code VARCHAR2(15) PRIMARY KEY,
    subject_name  VARCHAR2(100),
    professor     VARCHAR2(50)
);

CREATE TABLE grades (
    student_id    VARCHAR2(15),
    subject_code  VARCHAR2(15),
    sa NUMBER, ta NUMBER, sw NUMBER, rec NUMBER, fp NUMBER,
    me NUMBER, fe NUMBER,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES
students(student_id),
    CONSTRAINT fk_subject FOREIGN KEY (subject_code) REFERENCES
subjects(subject_code)
);

CREATE TABLE audits (
    log_id       NUMBER PRIMARY KEY,
    log_message  VARCHAR2(4000),
    log_date     DATE,
    db_user      VARCHAR2(100)
);
```


Outputs:

5. Source Codes

Files:

- **database_initialization.sql**: Does the entire setup process by executing table creation, constraints, and initial data population in the correct order.
- **delete_students.sql, delete_teachers.sql and delete_subjects.sql**: Allows the teachers to delete student, teachers, or subject records safely.
- **exit.sql**: Clears the terminal and displays a closing thank-you message when exiting the program.
- **functions_and_procedures.sql**: A consolidated library of PL/SQL blocks that handle core logic, including calculations for midterm and final grades, GWA, and pass/fail status.
- **insert_records.sql**: Preloads the database with sample data, including 6 subjects, 6 teacher accounts, and 100 randomized student records.
- **login_error.sql**: Provides feedback for failed login attempts and redirects users back to the main menu.
- **logout.sql**: Cleans up the session by undefining global variables and returning the user to the system entry point.
- **main.sql**: Serves as the application's entry point, routing users to teacher login, student login, developer info, or exit.
- **student_login.sql**: Handles student authentication and stores the logged-in student's ID in a global variable.
- **student_menu.sql**: Acts as the main navigation hub for students, offering options to view grades or return to the main menu.
- **student_view_grades.sql**: Fetches student records and calculates grades in real time using defined functions.
- **tables.sql**: Initializes the database by dropping and recreating teachers, students, subjects, and grades tables.
- **teacher_add_records.sql**: Allows teachers to input and save new student information into the database.
- **teacher_add_records_students.sql**: Used by teachers to register new students into the database.
- **teacher_add_records_subjects.sql**: Creates new subject entries in the database.
- **teacher_add_records_teachers.sql**: Adds new teacher records and assigns them to subjects.
- **teacher_delete_records.sql**: A sub-navigation menu that grants teachers the authority to select which type of records to remove from the system.
- **teacher_login.sql**: Manages teacher authentication and grants access to teacher-specific menus.
- **teacher_menu.sql**: Central navigation interface for teachers, allowing access to record management and grade updates.

- **teacher_update_records.sql**: Enables teachers to input or update student assessment scores.
- **teacher_view_records.sql**: A navigation sub-menu that allows teachers to select between viewing student records, teacher lists, or subject details.
- **teacher_view_records_students.sql**: Allows teachers to view student grades and filter the list from all, passing, and failing only students.
- **teacher_view_records_subjects.sql**: Allows teachers to view the list of subject codes, names, and the professors assigned to each subject.
- **teacher_view_records_teachers.sql**: Shows the list of all teachers in the system and the subject they teach.
- **triggers.sql**: Automates system actions such as checking grade range, calculating results, and logging student deletion logs.
- **view_dev_info.sql**: Displays information about the developers of the system.

Codes and Analysis:

environment settings

- **SET SERVEROUTPUT ON**
- **SET FEEDBACK OFF**
- **SET VERIFY OFF**
- **SET ECHO OFF**
- **SET HEADING OFF.**
- **SET PAGESIZE 0**
- **SET DEFINE ON**
- **SET TERMOUT ON**
- **CL SCR**

exit.sql

```
-- exit.sql

CL SCR
PROMPT
=====
=====
PROMPT .
EXIT
PROMPT
=====
=====
```

PROMPT

PROMPT Thank you!

Purpose: Provide visual feedback to the user after exiting the program.

functions_and_procedures.sql

```
-- functions_and_procedures.sql

CREATE OR REPLACE FUNCTION fn_calculate_midterm(
    p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_me NUMBER
) RETURN NUMBER IS
    v_avg NUMBER;
BEGIN
    v_avg := (p_sa + p_ta + p_sw + p_rec) / 4;
    RETURN ROUND((0.6 * v_avg) + (0.4 * p_me));
END;
/

CREATE OR REPLACE FUNCTION fn_calculate_final(
    p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_fp NUMBER, p_me
NUMBER, p_fe NUMBER
) RETURN NUMBER IS
    v_avg NUMBER;
BEGIN
    v_avg := (p_sa + p_ta + p_sw + p_rec + p_fp) / 5;
    RETURN ROUND((0.6 * v_avg) + (0.15 * p_me) + (0.25 * p_fe));
END;
/

CREATE OR REPLACE FUNCTION fn_calculate_gwa(p_mid IN NUMBER, p_fin IN NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN ROUND((p_mid + p_fin) / 2);
END;
/

CREATE OR REPLACE FUNCTION fn_get_status(p_gwa IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
    IF p_gwa >= 70 THEN
        RETURN 'PASSED';
```

```

        ELSE
            RETURN 'FAILED';
        END IF;
    END;
/

CREATE OR REPLACE PROCEDURE update_all_grades AS
BEGIN
    UPDATE grades
    SET me = fn_calculate_midterm(sa, ta, sw, rec, me);

    UPDATE grades
    SET fe = fn_calculate_final(sa, ta, sw, rec, fp, me, fe);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('All Grades have been updated successfully.');
```

```

END;
/

CREATE OR REPLACE PROCEDURE add_student (
    p_sid IN VARCHAR2,
    p_sec IN VARCHAR2
) AS
BEGIN
    INSERT INTO students (student_id, name, section)
    VALUES (TRIM(p_sid), 'NEW STUDENT', UPPER(TRIM(p_sec)));
END;
/

CREATE OR REPLACE PROCEDURE add_teacher(
    p_full_name IN VARCHAR2,
    p_scode     IN VARCHAR2
) AS
    v_first_init    VARCHAR2(2);
    v_last_name     VARCHAR2(50);
    v_generated_user VARCHAR2(20);
    v_generated_pass VARCHAR2(30);
    v_check_subj    NUMBER;
BEGIN
    v_first_init := LOWER(SUBSTR(p_full_name, 1, 1)) || LOWER(SUBSTR(p_full_name,
1, 1));
    v_last_name  := LOWER(SUBSTR(p_full_name, INSTR(p_full_name, ' ', -1) + 1));
    v_generated_user := v_first_init || v_last_name;
```

```

v_generated_pass := 'prof_' || v_generated_user;

SELECT COUNT(*) INTO v_check_subj FROM subjects WHERE subject_code =
UPPER(p_scode);

IF v_check_subj = 0 THEN
    DBMS_OUTPUT.PUT_LINE('[Error] Subject Code ' || p_scode || ' is
invalid.');
```

```

ELSE
    INSERT INTO teachers (username, name, password)
    VALUES (v_generated_user, p_full_name, v_generated_pass);

    UPDATE subjects
    SET professor = p_full_name
    WHERE subject_code = UPPER(p_scode);

    COMMIT;

DBMS_OUTPUT.PUT_LINE('=====');
    DBMS_OUTPUT.PUT_LINE('Professor Added Successfully.');
```

```

    DBMS_OUTPUT.PUT_LINE('Username : ' || v_generated_user);
    DBMS_OUTPUT.PUT_LINE('Password : ' || v_generated_pass);
    DBMS_OUTPUT.PUT_LINE('Assigned Subject : ' || UPPER(p_scode));

DBMS_OUTPUT.PUT_LINE('=====');
END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('[Error] Registration failed. Check name format or
duplicate user.');
```

```

        ROLLBACK;
END;
/

CREATE OR REPLACE PROCEDURE add_subject(
    p_scode IN VARCHAR2,
    p_sname IN VARCHAR2
) AS
    v_check NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_check FROM subjects WHERE subject_code =
UPPER(p_scode);
```



```
PROMPT
PROMPT [Error] The Username or Password you entered is incorrect.
PROMPT Go back to Main Menu?
PAUSE

@main.sql
```

Purpose: Provide visual feedback to the user after failing to authenticate before redirecting the user to the main menu.

logout.sql

```
-- logout.sql

UNDEFINE g_id
UNDEFINE Enter_Student_Number
UNDEFINE v_teacher_id

CL SCR

PROMPT Logged out successfully.
PAUSE Go back to Main Menu?

@main.sql
```

Purpose: Clears all global variables and credentials from the system's memory to ensure data security. It provides a visual logout confirmation and uses a PAUSE command to allow the user to acknowledge the exit before redirecting back to the main menu.

main.sql

```
-- main.sql

SET SERVEROUTPUT ON
SET FEEDBACK OFF
SET VERIFY OFF
SET ECHO OFF
SET HEADING OFF
SET PAGESIZE 0
```

```

SET DEFINE ON
SET TERMOUT ON
CL SCR

PROMPT
=====

=====
PROMPT .                                JVS STUDENT MANAGEMENT
SYSTEM .
PROMPT
=====

=====
PROMPT [1] Login as Teacher
PROMPT [2] Login as Student
PROMPT [3] View Developer Information
PROMPT [4] Exit
PROMPT
=====

=====
PROMPT

ACCEPT v_choice NUMBER PROMPT '[Main Menu] | Enter Choice (1-4): '

SET TERMOUT OFF
COLUMN v_run NEW_VALUE v_run

SELECT CASE
    WHEN &v_choice = 1 THEN 'teacher_login.sql'
    WHEN &v_choice = 2 THEN 'student_login.sql'
    WHEN &v_choice = 3 THEN 'view_dev_info.sql'
    WHEN &v_choice = 4 THEN 'exit.sql'
    ELSE 'main.sql'
END AS v_run
FROM dual;

SET TERMOUT ON
@&v_run

```

Purpose: Displays the system's main menu, prompts the user for a valid choice (teacher login, student login, developer info, or exit), and dynamically runs the corresponding script based on the selection.

student_login.sql

```
-- student_login.sql

SET FEEDBACK OFF
SET VERIFY OFF
SET DEFINE ON
CL SCR

UNDEFINE input_id
UNDEFINE input_pass
UNDEFINE v_next

PROMPT
=====
=====
PROMPT .                                STUDENT
LOGIN                                  .
PROMPT
=====
=====

ACCEPT input_id PROMPT '[Student Login] | Enter Student Number: '
ACCEPT input_pass PROMPT '[Student Login] | Enter Password: ' HIDE

VARIABLE v_auth_count NUMBER
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM students
    WHERE student_id = '&input_id' AND password = '&input_pass';

    :v_auth_count := v_count;
END;
/

SET TERMOUT OFF
COLUMN input_id NEW_VALUE g_id
SELECT '&input_id' AS input_id FROM dual;
DEFINE g_id = '&input_id'
SET TERMOUT ON
```

```

COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE
        WHEN :v_auth_count = 1 THEN 'student_menu.sql'
        ELSE 'login_error.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@&v_next

```

Purpose: Resets previous input, prompts the student for ID and password, verifies credentials against the database, stores the session ID for reuse, and directs the user to either the student menu or an error page based on authentication results.

student_menu.sql

```

-- student_menu.sql

CL SCR

PROMPT
=====
=====
PROMPT .                                STUDENT
MENU                                .
PROMPT
=====
=====
PROMPT [1] View My Grades
PROMPT [2] Logout and Go Back to Main Menu
PROMPT
=====
=====
PROMPT STUDENT NUMBER: &g_id
PROMPT

ACCEPT v_choice NUMBER PROMPT '[Student Menu] | Enter Choice (1-2): '

SET TERMOUT OFF

```

```

COLUMN init_pos NEW_VALUE v_pos
SELECT '1' AS init_pos FROM dual;

COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE
        WHEN &v_choice = 1 THEN 'student_view_grades.sql'
        WHEN &v_choice = 2 THEN 'logout.sql'
        ELSE 'student_menu.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@&v_next

```

Purpose: Displays the student menu with options to view grades or logout, prompts for a valid choice, and dynamically runs the corresponding script based on the student's selection.

tables.sql

```

-- tables.sql

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE grades CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE subjects CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE students CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE teachers CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE audits CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

CREATE TABLE teachers (
    username    VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    password    VARCHAR2(20)
);

CREATE TABLE students (
    student_id  VARCHAR2(15) PRIMARY KEY,
    name        VARCHAR2(100),
    section     VARCHAR2(10),

```

```

    password    VARCHAR2(20)
);

CREATE TABLE subjects (
    subject_code VARCHAR2(15) PRIMARY KEY,
    subject_name VARCHAR2(100),
    professor    VARCHAR2(50)
);

CREATE TABLE grades (
    student_id   VARCHAR2(15),
    subject_code VARCHAR2(15),
    sa NUMBER, ta NUMBER, sw NUMBER, rec NUMBER, fp NUMBER,
    me NUMBER, fe NUMBER,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES
students(student_id),
    CONSTRAINT fk_subject FOREIGN KEY (subject_code) REFERENCES
subjects(subject_code)
);

CREATE TABLE audits (
    log_id       NUMBER PRIMARY KEY,
    log_message  VARCHAR2(4000),
    log_date     DATE,
    db_user      VARCHAR2(100)
);

```

Purpose: Creates the core tables for the system (teachers, students, subjects, grades, and audits) with primary and foreign key constraints, and ensures any existing tables are dropped first to start with a clean database structure.

insert_records.sql

```

-- insert_records.sql

INSERT INTO subjects VALUES ('CS0021', 'DISCRETE STRUCTURES', 'Marilyn Sanchez');
INSERT INTO subjects VALUES ('CS0017', 'OPERATING SYSTEM', 'Jeneffer
Sabonsolin');
INSERT INTO subjects VALUES ('CS0010', 'FUNDAMENTALS OF ANALYTICS', 'Anthony
Aquino');
INSERT INTO subjects VALUES ('CS0005', 'DATABASE SYSTEMS', 'Fanny Almeniana');
INSERT INTO subjects VALUES ('CS0007', 'ALGORITHM', 'Justine Jude Pura');

```

```
INSERT INTO subjects VALUES ('CCS0101', 'DESIGN THINKING', 'May Florence San Pablo');
```

```
INSERT INTO teachers VALUES ('mmsanchez', 'Marilyn Sanchez', 'prof_mmsanchez');
```

```
INSERT INTO teachers VALUES ('jjsabonsolin', 'Jeneffer Sabonsolin', 'prof_jjsabonsolin');
```

```
INSERT INTO teachers VALUES ('aaaquino', 'Anthony Aquino', 'prof_aaaquino');
```

```
INSERT INTO teachers VALUES ('ffalmeniana', 'Fanny Almeniana', 'prof_ffalmeniana');
```

```
INSERT INTO teachers VALUES ('jjpura', 'Justine Jude Pura', 'prof_jjpura');
```

```
INSERT INTO teachers VALUES ('mmsanpablo', 'May Florence San Pablo', 'prof_mmsanpablo');
```

```
DECLARE
```

```
    TYPE t_name IS TABLE OF VARCHAR2(50);
```

```
    v_fn t_name := t_name(
```

```
'Aaron','Abigail','Adrian','Aileen','Albert','Alex','Alicia','Alvin','Amanda','Andrew',
```

```
'Angela','Angelo','Anita','Anthony','Arlene','Arnold','Arthur','Beatrice','Benjie',  
'Bernadette',
```

```
'Bethany','Bianca','Bobby','Brenda','Bryan','Camille','Carl','Carmen','Cedric','Celine',
```

```
'Charles','Chloe','Chris','Clara','Claude','Cynthia','Daisy','Daniel','Dante','Daphne',
```

```
'David','Debbie','Dennis','Diana','Diego','Dominic','Donna','Edgar','Edith','Eduardo',
```

```
'Edwin','Elaine','Eleanor','Elias','Elisa','Elizabeth','Ellen','Elmer','Elsa','Emanuel',
```

```
'Eric','Erica','Esther','Eugene','Eunice','Eva','Evelyn','Felix','Ferdinand','Fernando',
```

```
'Fiona','Flora','Florence','Francis','Fred','Gabriel','Gemma','George','Gerald','Gina',
```

```
'Gloria','Grace','Gregorio','Hannah','Harold','Hazel','Hector','Helen','Henry','Herbert',
```

```

'Herman','Homer','Ida','Ignacio','Imelda','Irene','Iris','Isaac','Isabel','Isidro',
);

v_ln t_name := t_name(

'Abad','Abalos','Abaya','Abril','Agoncillo','Aguinaldo','Alcasid','Almazan','Alme
da','Alonzo',

'Alvarado','Alvarez','Amante','Andrada','Angeles','Antipolo','Apostol','Aragon','
Araneta','Arceo',

'Arellano','Arnaiz','Arroyo','Asis','Atienza','Austria','Balagtas','Baluyot','Ban
ayo','Barretto',

'Basco','Belmonte','Benitez','Bernardo','Blanco','Bonifacio','Borja','Buenaventur
a','Buencamino','Burgos',

'Caballero','Cabanatuan','Cabrera','Cacho','Calaguas','Calderon','Calungsod','Cam
pos','Canlas','Caparas',

'Carreon','Castillo','Castro','Catacutan','Cayetano','Cervantes','Chu','Claudio',
'Co','Concepcion',

'Constantino','Coronel','Cortez','Crisostomo','Cruz','Cuenco','Cunanan','Custodio
','Dagohoy','Dandan',
'David','Dayrit','De Guzman','De Jesus','De Leon','De Mesa','Del
Rosario','Delos Reyes','Delos Santos','Diamante',

'Diaz','Dilangalen','Dimaculangan','Dimalanta','Dimasalang','Diokno','Domingo','D
ominguez','Dorado','Dumlao',

'Echeverria','Ejercito','Encarnacion','Enrile','Enriquez','Escudero','Esguerra','
Espina','Espinosa','Espiritu'
);

v_sid VARCHAR2(15);
BEGIN
FOR i IN 1..100 LOOP
v_sid := '2024' || TRUNC(DBMS_RANDOM.VALUE(10000,99999));

INSERT INTO students VALUES (

```

```

        v_sid,
        UPPER(v_fn(i) || ' ' || v_ln(i)),
        'TN2' || TRUNC(DBMS_RANDOM.VALUE(1,10)),
        v_sid
    );

    FOR r_sub IN (SELECT subject_code FROM subjects) LOOP
        INSERT INTO grades VALUES (
            v_sid,
            r_sub.subject_code,
            TRUNC(DBMS_RANDOM.VALUE(50,98)),
            TRUNC(DBMS_RANDOM.VALUE(50,98)),
            TRUNC(DBMS_RANDOM.VALUE(50,98)),
            TRUNC(DBMS_RANDOM.VALUE(50,98)),
            TRUNC(DBMS_RANDOM.VALUE(50,98)),
            TRUNC(DBMS_RANDOM.VALUE(50,98))
        );
    END LOOP;
END LOOP;
COMMIT;
END;
/

PROMPT Records Inserted Successfully.

```

Purpose: Inserts sample data into subjects and teachers, generates 100 student records with randomized sections and placeholder passwords, and automatically populates the grades table with random scores for all students and subjects, providing a ready-to-use dataset for testing and system functionality.

student_view_grades.sql

```

-- student_view_grades.sql

SET SERVEROUTPUT ON
SET FEEDBACK OFF
SET VERIFY OFF

COLUMN col_pos NEW_VALUE v_pos
SELECT NVL('&v_pos', '1') AS col_pos FROM dual;

```

```

CL SCR
PROMPT
=====
=====
PROMPT .                                VIEW
GRADES                                  .
PROMPT
=====
=====

DECLARE
    v_id          VARCHAR2(15) := '&g_id';
    v_current      NUMBER := TO_NUMBER('&v_pos');
    v_total        NUMBER;

    v_midterm      NUMBER;
    v_final        NUMBER;
    v_gwa          NUMBER;
    v_status       VARCHAR2(15);

    CURSOR c_item IS
        SELECT * FROM (
            SELECT s.name, s.section, sub.subject_code, sub.subject_name,
sub.professor,
                g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe,
                ROW_NUMBER() OVER (ORDER BY sub.subject_code) as rn
            FROM students s
            JOIN grades g ON s.student_id = g.student_id
            JOIN subjects sub ON g.subject_code = sub.subject_code
            WHERE s.student_id = v_id
        ) WHERE rn = v_current;

    r c_item%ROWTYPE;
BEGIN
    SELECT COUNT(*) INTO v_total FROM grades WHERE student_id = v_id;

    OPEN c_item;
    FETCH c_item INTO r;

    IF c_item%FOUND THEN
        v_midterm := fn_calculate_midterm(r.sa, r.ta, r.sw, r.rec, r.me);
        v_final   := fn_calculate_final(r.sa, r.ta, r.sw, r.rec, r.fp, r.me,
r.fe);

```



```

        v_gwa      := fn_calculate_gwa(v_midterm, v_final);
        v_status   := fn_get_status(v_gwa);

        DBMS_OUTPUT.PUT_LINE('STUDENT NO : ' || v_id);
        DBMS_OUTPUT.PUT_LINE('NAME       : ' || UPPER(r.name));
        DBMS_OUTPUT.PUT_LINE('SECTION   : ' || r.section || ' | Subject ' ||
v_current || ' of ' || v_total);

DBMS_OUTPUT.PUT_LINE('=====
=====');
        DBMS_OUTPUT.PUT_LINE('COURSE : ' || r.subject_code || ' - ' ||
r.subject_name);
        DBMS_OUTPUT.PUT_LINE('PROF    : ' || r.professor);

DBMS_OUTPUT.PUT_LINE('=====
=====');

DBMS_OUTPUT.PUT_LINE(RPAD('SA',10)||RPAD('TA',10)||RPAD('SW',10)||RPAD('REC',10)|
|RPAD('FP',10)||RPAD('ME',10)||'FE');

DBMS_OUTPUT.PUT_LINE(RPAD(r.sa,10)||RPAD(r.ta,10)||RPAD(r.sw,10)||RPAD(r.rec,10)|
|RPAD(r.fp,10)||RPAD(r.me,10)||r.fe);

DBMS_OUTPUT.PUT_LINE('=====
=====');

DBMS_OUTPUT.PUT_LINE(RPAD('MIDTERM',20)||RPAD('FINAL',20)||RPAD('GWA',20)||'STATU
S');
        DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(v_midterm,
'990'),20)||RPAD(TO_CHAR(v_final, '990'),20)||RPAD(TO_CHAR(v_gwa,
'990'),20)||v_status);

DBMS_OUTPUT.PUT_LINE('=====
=====');
        ELSE
            DBMS_OUTPUT.PUT_LINE('No more subjects to display. ');
        END IF;
        CLOSE c_item;
END;
/

PROMPT [1] Next Subject
PROMPT [2] Previous Subject
PROMPT [3] Go back to Student Menu

```

```

ACCEPT v_nav PROMPT 'What would you like to do next? '

SET TERMOUT OFF
COLUMN v_pos NEW_VALUE v_pos
COLUMN next_script NEW_VALUE v_script

SELECT
    CASE '&v_nav'
        WHEN '1' THEN TO_CHAR(&v_pos + 1)
        WHEN '2' THEN TO_CHAR(CASE WHEN &v_pos > 1 THEN &v_pos - 1 ELSE 1 END)
        ELSE TO_CHAR(&v_pos)
    END AS v_pos,
    CASE '&v_nav'
        WHEN '3' THEN 'student_menu.sql'
        ELSE 'student_view_grades.sql'
    END AS next_script
FROM dual;

UNDEFINE v_nav
SET TERMOUT ON

@&v_script

```

Purpose: Retrieves and displays a student's grades for each subject, calculates midterm, final, GWA, and pass/fail status, and allows navigation between subjects or returning to the student menu.

teacher_login.sql

```

-- teacher_login.sql
teacher_1
SET FEEDBACK OFF
SET VERIFY OFF
SET DEFINE ON
CL SCR

UNDEFINE input_id
UNDEFINE input_pass
UNDEFINE v_next

```

```

PROMPT
=====
=====
PROMPT .                                TEACHER
LOGIN                                  .
PROMPT
=====
=====

ACCEPT input_id PROMPT '[Teacher Login] | Enter Username: '
ACCEPT input_pass PROMPT '[Teacher Login] | Enter Password: ' HIDE

VARIABLE v_auth_count NUMBER
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM teachers
    WHERE username = '&input_id' AND password = '&input_pass';

    :v_auth_count := v_count;
END;
/

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_next

SELECT
    CASE
        WHEN :v_auth_count = 1 THEN 'teacher_menu.sql'
        ELSE 'login_error.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@@&v_next

```

Purpose: Prompts the teacher for username and password, verifies credentials against the teachers table, and directs the user to either the teacher menu or a login error page based on the authentication result.

teacher_menu.sql

```
-- teacher_menu.sql

SET FEEDBACK OFF
SET VERIFY OFF
CL SCR

PROMPT
=====
=====
PROMPT .                                TEACHER
MENU .
PROMPT
=====
=====
PROMPT [1] View Records
PROMPT [2] Add Records
PROMPT [3] Update Records
PROMPT [4] Delete Records
PROMPT [5] Logout and Go back to Main Menu
PROMPT
=====
=====

PROMPT LOGGED IN AS: &input_id
PROMPT

ACCEPT v_choice NUMBER PROMPT '[Teacher Menu] | Enter Choice (1-5): '

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_run
SELECT
    CASE
        WHEN &v_choice = 1 THEN 'teacher_view_records.sql'
        WHEN &v_choice = 2 THEN 'teacher_add_records.sql'
        WHEN &v_choice = 3 THEN 'teacher_update_records.sql'
        WHEN &v_choice = 4 THEN 'teacher_delete_records.sql'
        WHEN &v_choice = 5 THEN 'logout.sql'
        ELSE 'teacher_menu.sql'
    END AS next_scr
FROM dual;
```

```
SET TERMOUT ON
```

```
@&v_run
```

Purpose: Displays the teacher menu with options to view, add, update, or delete records, or logout, prompts for a valid choice, and dynamically runs the corresponding script based on the teacher's selection.

teacher_view_records.sql

```
-- teacher_view_records.sql

SET FEEDBACK OFF
SET VERIFY OFF
SET SERVEROUTPUT ON
CL SCR

PROMPT
=====
=====
PROMPT .                                VIEW
RECORDS                                .
PROMPT
=====
=====
PROMPT [1] View Students
PROMPT [2] View Professors
PROMPT [3] View Subjects
PROMPT [4] Go back to Teacher Menu
PROMPT
=====
=====

ACCEPT v_view_choice NUMBER PROMPT '[View Records] | Enter Choice (1-4): '

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_next

SELECT
    CASE
        WHEN &v_view_choice = 1 THEN 'teacher_view_records_students.sql'
        WHEN &v_view_choice = 2 THEN 'teacher_view_records_teachers.sql'
```

```

        WHEN &v_view_choice = 3 THEN 'teacher_view_records_subjects.sql'
        WHEN &v_view_choice = 4 THEN 'teacher_menu.sql'
        ELSE 'teacher_view_records.sql'
    END AS next_scr
FROM dual;

SET TERMOUT ON

@&v_next

```

Purpose: Allows the teacher to choose whether to view students, teachers, or subjects, or return to the teacher menu, and dynamically runs the corresponding script based on the selection.

teacher_view_records_students.sql

```

SET VERIFY OFF
SET SERVEROUTPUT ON
SET FEEDBACK OFF
CLEAR SCREEN

PROMPT
=====
=====
PROMPT .                VIEW STUDENT
RECORDS                .
PROMPT
=====
=====

ACCEPT v_section PROMPT '[View Student Records] | Enter Section (TN21-TN29): '

PROMPT
PROMPT [1] View All Students
PROMPT [2] View Passing Students
PROMPT [3] View Failed Students
PROMPT [4] Go back to View Records Menu
PROMPT
=====
=====
ACCEPT v_filter PROMPT '[View Student Records] | Enter Choice (1-4): '

```

```

DECLARE
    CURSOR c_filtered_students IS
        SELECT
            s.student_id,
            s.name,
            s.section,
            NVL(ROUND(AVG(fn_calculate_gwa(
                fn_calculate_midterm(g.sa, g.ta, g.sw, g.rec, g.me),
                fn_calculate_final(g.sa, g.ta, g.sw, g.rec, g.fp, g.me, g.fe)
            )), 0) as overall_gwa
        FROM students s
        LEFT JOIN grades g ON s.student_id = g.student_id
        WHERE UPPER(TRIM(s.section)) = UPPER(TRIM('&v_section'))
        GROUP BY s.student_id, s.name, s.section
        ORDER BY s.name ASC;

    v_found    BOOLEAN := FALSE;
    v_choice   NUMBER  := TO_NUMBER('&v_filter');
    v_status   VARCHAR2(10);
BEGIN
    IF v_choice = 4 THEN
        GOTO end_script;
    END IF;

    DBMS_OUTPUT.PUT_LINE(CHR(10));
    DBMS_OUTPUT.PUT_LINE(RPAD('ID', 15) || RPAD('NAME', 30) || RPAD('GWA', 10) ||
    'STATUS');

    DBMS_OUTPUT.PUT_LINE('=====
    =====');

    FOR r IN c_filtered_students LOOP

        IF r.overall_gwa >= 70 THEN
            v_status := 'PASSED';
        ELSE
            v_status := 'FAILED';
        END IF;

        IF (v_choice = 1) OR
            (v_choice = 2 AND v_status = 'PASSED') OR
            (v_choice = 3 AND v_status = 'FAILED') THEN

```

```

        DBMS_OUTPUT.PUT_LINE(
            RPAD(r.student_id, 15) ||
            RPAD(UPPER(r.name), 30) ||
            RPAD(TO_CHAR(r.overall_gwa), 10) ||
            v_status
        );
        v_found := TRUE;
    END IF;
END LOOP;

IF NOT v_found THEN
    DBMS_OUTPUT.PUT_LINE(' ');
    DBMS_OUTPUT.PUT_LINE('No records found for this filter/section.');
```

=====

```

END IF;

DBMS_OUTPUT.PUT_LINE('=====');

<<end_script>>
NULL;
END;
/

PAUSE Go back to previous menu?

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE
        WHEN '&v_filter' = '4' THEN 'teacher_view_records.sql'
        ELSE 'teacher_view_records_students.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@&v_next

```

Purpose: Prompts the teacher to select a section and filter (all, passing, or failing students), calculates each student's overall GWA and pass/fail status, displays the filtered list, and allows navigation back to the previous menu.

teacher_add_records.sql

```
-- teacher_add_records.sql

SET FEEDBACK OFF
SET VERIFY OFF
CL SCR

PROMPT
=====
=====
PROMPT .                                ADD
RECORDS                                .
PROMPT
=====
=====
PROMPT [1] Add New Student
PROMPT [2] Add New Professor
PROMPT [3] Add New Subject
PROMPT [4] Go back to Teacher Menu
PROMPT
=====
=====

ACCEPT v_add_choice NUMBER PROMPT '[Add Records] | Enter Choice (1-4): '

SET TERMOUT OFF
COLUMN next_scr NEW_VALUE v_run
SELECT
    CASE
        WHEN &v_add_choice = 1 THEN 'teacher_add_records_students.sql'
        WHEN &v_add_choice = 2 THEN 'teacher_add_records_teachers.sql'
        WHEN &v_add_choice = 3 THEN 'teacher_add_records_subjects.sql'
        ELSE 'teacher_menu.sql'
    END AS next_scr
FROM dual;
SET TERMOUT ON

@&v_run
```

Purpose: Displays options for the teacher to add a new student, professor, or subject, prompts for a choice, and dynamically runs the corresponding script based on the selection.

teacher_add_records_students.sql

```
SET FEEDBACK OFF
SET VERIFY OFF
SET SERVEROUTPUT ON SIZE UNLIMITED
CL SCR

PROMPT
=====
=====
PROMPT .                                ADD STUDENT
RECORD .
PROMPT
=====
=====

ACCEPT v_sid PROMPT '[Add Student Record] | Enter Student Number: '
ACCEPT v_name PROMPT '[Add Student Record] | Enter Full Name: '
ACCEPT v_sec PROMPT '[Add Student Record] | Enter Section (TN21-TN29): '

DECLARE
    v_id      VARCHAR2(50) := UPPER(TRIM('&v_sid'));
    v_name     VARCHAR2(100) := UPPER(TRIM('&v_name'));
    v_section  VARCHAR2(10) := UPPER(TRIM('&v_sec'));
    v_check    NUMBER;
BEGIN
    IF v_section NOT LIKE 'TN2_' OR SUBSTR(v_section, 4, 1) NOT BETWEEN '1' AND
'9' THEN
        DBMS_OUTPUT.PUT_LINE(CHR(10));

DBMS_OUTPUT.PUT_LINE('=====
=====');
        DBMS_OUTPUT.PUT_LINE('[Error] Invalid Section. Only TN21 to TN29 are
allowed.');
```

```
DBMS_OUTPUT.PUT_LINE('=====
=====');
    ELSE
        SELECT COUNT(*) INTO v_check
        FROM students
```

```

        WHERE TRIM(student_id) = v_id;

        IF v_check > 0 THEN
            DBMS_OUTPUT.PUT_LINE(CHR(10));

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
            DBMS_OUTPUT.PUT_LINE('[Error] Student ID ' || v_id || ' already
exists.');
```

```

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
        ELSE
            INSERT INTO students (student_id, name, section)
            VALUES (v_id, v_name, v_section);
            COMMIT;

            DBMS_OUTPUT.PUT_LINE(CHR(10));

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
            DBMS_OUTPUT.PUT_LINE('Student ' || v_id || ' in section ' ||
v_section || ' added successfully.');
```

```

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
        END IF;
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(CHR(10));

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
        DBMS_OUTPUT.PUT_LINE('[Error] ' || SQLERRM);

DBMS_OUTPUT.PUT_LINE('=====');
DBMS_OUTPUT.PUT_LINE('=====');
END;
/
```

```
PAUSE Go back to previous menu?
```

```
UNDEFINE v_sid
```

```
UNDEFINE v_sec
```

```
@teacher_menu.sql
```

Purpose: Prompts the teacher to enter a new student's ID, name, and section, validates the section and checks for duplicate IDs, inserts the student into the students table if valid, and displays success or error messages.

teacher_add_records_teachers.sql

```
-- teacher_add_records_professors.sql
```

```
SET FEEDBACK OFF
```

```
SET VERIFY OFF
```

```
SET SERVEROUTPUT ON
```

```
CL SCR
```

```
PROMPT
```

```
=====
```

```
=====
```

```
PROMPT .                                ADD PROFESSOR
```

```
RECORD .
```

```
PROMPT
```

```
=====
```

```
=====
```

```
ACCEPT v_full_name PROMPT '[Add Professor Records] | Enter Professor Full Name: '
```

```
PROMPT
```

```
=====
```

```
=====
```

```
PROMPT AVAILABLE SUBJECTS TO TEACH:
```

```
COLUMN subject_code FORMAT A12 HEADING 'CODE'
```

```
COLUMN subject_name FORMAT A30 HEADING 'SUBJECT TITLE'
```

```
SELECT subject_code, subject_name FROM subjects;
```

```
PROMPT
```

```
=====
```

```
=====
```

```

ACCEPT v_scode PROMPT '[Add Professor Records] | Enter Subject Code to assign: '

DECLARE
    v_input_name VARCHAR2(100) := '&v_full_name';
    v_first_init VARCHAR2(2);
    v_last_name VARCHAR2(50);
    v_generated_user VARCHAR2(20);
    v_generated_pass VARCHAR2(30);
    v_check_subj NUMBER;
BEGIN
    add_professor('&v_full_name', '&v_scode');
END;
/

PAUSE Go back to Add Records Menu?

@teacher_add_records.sql

```

Purpose: Prompts the teacher for a professor's full name and the subject to assign, then calls a procedure to generate the professor's username and password, insert the record into the teachers table, and link the professor to the selected subject.

teacher_add_records_subjects.sql

```

-- teacher_add_records_subjects.sql

SET FEEDBACK OFF
SET VERIFY OFF
SET SERVEROUTPUT ON
CL SCR

PROMPT
=====
=====
PROMPT .                                ADD SUBJECT
RECORD                                .
PROMPT
=====
=====

ACCEPT v_scode PROMPT '[Add Subject Record] | Enter Subject Code: '
ACCEPT v_sname PROMPT '[Add Subject Record] | Enter Subject Name: '

```

```

DECLARE
    v_check NUMBER;
BEGIN
    add_subject('&v_scode', '&v_sname');
END;
/

PAUSE Go back to previous menu?

@teacher_add_records.sql

```

Purpose: Prompts the teacher for a subject code and name, then calls a procedure to insert the subject into the subjects table, initialize placeholder grades for all students, and display success or error messages.

teacher_update_records.sql

```

-- teacher_update_records.sql

SET FEEDBACK OFF
SET VERIFY OFF
SET SERVEROUTPUT ON
CL SCR

PROMPT
=====
=====
PROMPT .                                UPDATE
RECORDS                                .
PROMPT
=====
=====

COLUMN student_id FORMAT A15 HEADING 'ID NUMBER'
COLUMN name          FORMAT A30 HEADING 'STUDENT NAME'
SELECT student_id, name FROM students ORDER BY name;

PROMPT
ACCEPT v_sid PROMPT '[Update Records] | Enter Student ID: '

```

```

COLUMN name NEW_VALUE v_sname DEFAULT 'NOT_FOUND'
SET TERMOUT OFF
SELECT name FROM students WHERE student_id = '&v_sid';
SET TERMOUT ON

COLUMN action_cmd NEW_VALUE v_run_script
SET TERMOUT OFF
SELECT CASE
    WHEN '&v_sname' = 'NOT_FOUND' THEN '@teacher_update_records.sql'
    ELSE 'PROMPT ID Verified...'
END AS action_cmd
FROM dual;
SET TERMOUT ON

BEGIN
    IF '&v_sname' = 'NOT_FOUND' THEN
        DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> ERROR: Student ID [&v_sid] not
found!');
        DBMS_OUTPUT.PUT_LINE('>>> Restarting update process... Please wait.');
```

END IF;

```

END;
/

SET TERMOUT OFF
COLUMN undef_cmd NEW_VALUE v_undef
SELECT DECODE('&v_sname', 'NOT_FOUND', 'UNDEFINE v_sid', 'PROMPT Continuing...')
AS undef_cmd FROM dual;
SET TERMOUT ON

&v_undef
&v_run_script

PROMPT STUDENT NAME      : &v_sname
PROMPT STUDENT NUMBER   : &v_sid
PROMPT
PROMPT ENROLLED SUBJECTS
PROMPT

=====
=====
SELECT subject_code FROM grades WHERE student_id = '&v_sid';
PROMPT

=====
=====

```

```
ACCEPT v_scode PROMPT '[Update Records] | Enter Subject Code to Update: '
```

```
PROMPT
```

```
PROMPT CURRENT GRADES FOR &v_scode
```

```
PROMPT
```

```
=====
=====
```

```
COLUMN sa    FORMAT 999
```

```
COLUMN ta    FORMAT 999
```

```
COLUMN sw    FORMAT 999
```

```
COLUMN rec   FORMAT 999
```

```
COLUMN me    FORMAT 999
```

```
COLUMN fp    FORMAT 999
```

```
COLUMN fe    FORMAT 999
```

```
SELECT sa, ta, sw, rec, me, fp, fe
```

```
FROM grades
```

```
WHERE student_id = '&v_sid' AND UPPER(subject_code) = UPPER('&v_scode');
```

```
PROMPT
```

```
=====
=====
```

```
ACCEPT v_sa  NUMBER PROMPT 'New SA  : '
```

```
ACCEPT v_ta  NUMBER PROMPT 'New TA  : '
```

```
ACCEPT v_sw  NUMBER PROMPT 'New SW  : '
```

```
ACCEPT v_rec NUMBER PROMPT 'New REC : '
```

```
ACCEPT v_me  NUMBER PROMPT 'New ME  : '
```

```
ACCEPT v_fp  NUMBER PROMPT 'New FP  : '
```

```
ACCEPT v_fe  NUMBER PROMPT 'New FE  : '
```

```
BEGIN
```

```
    UPDATE grades
```

```
    SET sa = &v_sa, ta = &v_ta, sw = &v_sw, rec = &v_rec,
```

```
        me = &v_me, fp = &v_fp, fe = &v_fe
```

```
    WHERE student_id = '&v_sid' AND UPPER(subject_code) = UPPER('&v_scode');
```

```
    IF SQL%ROWCOUNT > 0 THEN
```

```
        COMMIT;
```

```
        DBMS_OUTPUT.PUT_LINE(CHR(10) || 'Grades successfully updated.');
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> [Error] Subject Code not found for  
this student.');
```



```

        END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> [Error] ' || SQLERRM);
END;
/

PROMPT
PROMPT UPDATED GRADES SUMMARY
PROMPT
=====
=====
SELECT
    sa, ta, sw, rec, me, fp, fe,
    ROUND(fn_calculate_gwa(
        fn_calculate_midterm(sa, ta, sw, rec, me),
        fn_calculate_final(sa, ta, sw, rec, fp, me, fe)
    )) AS "NEW_GWA"
FROM grades
WHERE student_id = '&v_sid' AND UPPER(subject_code) = UPPER('&v_scode');
PROMPT
=====
=====

PROMPT
PAUSE Press ENTER to return to Teacher Menu...

UNDEFINE v_sid
UNDEFINE v_sname
UNDEFINE v_scode
UNDEFINE v_sa
UNDEFINE v_ta
UNDEFINE v_sw
UNDEFINE v_rec
UNDEFINE v_me
UNDEFINE v_fp
UNDEFINE v_fe
UNDEFINE v_run_script
UNDEFINE v_undef

@teacher_menu.sql

```

Purpose: Allows the teacher to select a student and subject, verify the student ID, view current grades, input new scores, update the grades table, calculate the new GWA, display a summary, and handle errors or invalid entries.

teacher_delete_records.sql

```
-- teacher_delete_records.sql

SET FEEDBACK OFF
SET VERIFY OFF
CL SCR

PROMPT
=====
=====
PROMPT .                                DELETE
RECORDS                                .
PROMPT
=====
=====
PROMPT [1] Delete Student Record
PROMPT [2] Delete Professor Record
PROMPT [3] Delete Subject Record
PROMPT [4] Go back to Teacher Menu
PROMPT
=====
=====
ACCEPT v_del_choice NUMBER PROMPT '[Delete Records] | Enter Choice (1-4): '

COLUMN next_scr NEW_VALUE v_next
SELECT
    CASE '&v_del_choice'
        WHEN '1' THEN 'delete_students.sql'
        WHEN '2' THEN 'delete_teachers.sql'
        WHEN '3' THEN 'delete_subjects.sql'
        WHEN '4' THEN 'teacher_menu.sql'
        ELSE 'teacher_delete_records.sql'
    END AS next_scr
FROM dual;

@&v_next
```

Purpose: Displays options to delete a student, professor, or subject record, prompts the teacher for a choice, and dynamically runs the corresponding deletion script or returns to the teacher menu.

triggers.sql

```
-- triggers.sql

CREATE OR REPLACE TRIGGER trg_check_grade_range
BEFORE INSERT OR UPDATE ON grades
FOR EACH ROW
BEGIN
    IF :NEW.sa > 100 OR :NEW.ta > 100 OR :NEW.sw > 100 OR
        :NEW.rec > 100 OR :NEW.me > 100 OR :NEW.fe > 100 THEN
        RAISE_APPLICATION_ERROR(-20001, '[Trigger] Grades cannot be higher than
100.');
```

```
    END IF;
END;
/

CREATE OR REPLACE TRIGGER trg_auto_calc_results
BEFORE UPDATE ON grades
FOR EACH ROW
DECLARE
    v_mid NUMBER;
    v_fin NUMBER;
BEGIN
    v_mid := fn_calculate_midterm(:NEW.sa, :NEW.ta, :NEW.sw, :NEW.rec, :NEW.me);
    v_fin :=
fn_calculate_final(:NEW.sa, :NEW.ta, :NEW.sw, :NEW.rec, :NEW.fp, :NEW.me, :NEW.fe
);

    DBMS_OUTPUT.PUT_LINE('[Trigger] Recalculated results for Student. '
|| :NEW.student_id);
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002, '[Trigger] Error. Check if function
parameters match.');
```

```
END;
/

CREATE OR REPLACE TRIGGER trg_audit_student_delete
```

```

AFTER DELETE ON students
FOR EACH ROW
BEGIN
    INSERT INTO audits (log_message, log_date, db_user)
    VALUES ('[Trigger] Deleted Student ID: ' || :OLD.student_id, SYSDATE, USER);
END;
/

```

Purpose: Enforces grade rules and integrity, automatically recalculates midterm/final results whenever a grade row is updated, and records an audit log entry whenever a student record is deleted.

view_dev_info.sql

```

-- view_dev_info.sql

SET SERVEROUTPUT ON
SET FEEDBACK OFF
CL SCR

PROMPT
=====
=====
PROMPT .                                VIEW DEVELOPER
INFORMATION                               .
PROMPT
=====
=====
PROMPT
PROMPT Project Name : JVS Student Management Record System with PL/SQL
Implementation
PROMPT Group Name   : Share Q Lang(SQL)
PROMPT Developers   :
PROMPT Vince Carlo I. Lachica
PROMPT Janelle P. Facto
PROMPT Rasheed L. Librorania
PROMPT Description  : A PL/SQL-based system for managing and viewing
student academic records.
PROMPT
=====
=====

```

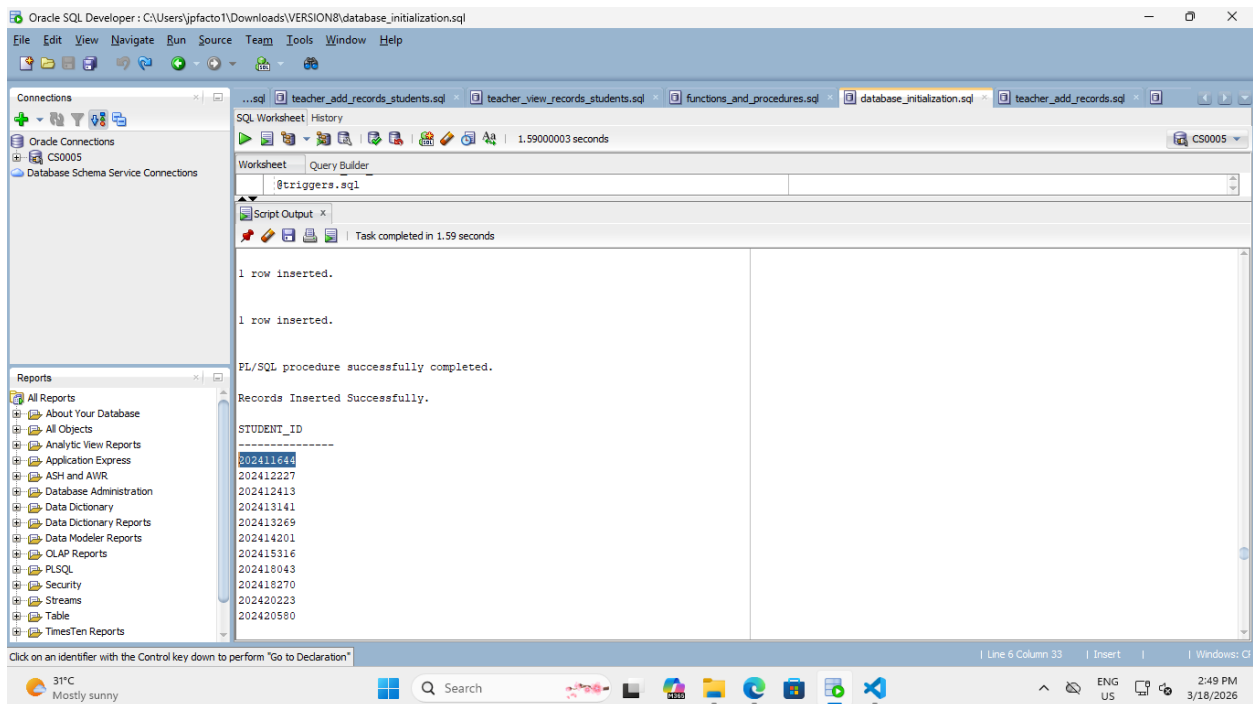
```
PROMPT
PAUSE Go back to Main Menu?

@main.sql
```

Purpose: Provide an option to view details about the program and developers.

6. Screenshots of Execution

Database initialization: Screenshot showing tables created successfully.



Insert records: Screenshot showing sample inserted rows.

```
Script Output x
Task completed in 1.59 seconds

STUDENT_ID
-----
202490724
202490767
202491038
202491629
202492274
202492828
202494195
202495806
202498850
202498968
202499161

STUDENT_ID
-----
202499479

100 rows selected.

USERNAME      PASSWORD
-----
mmsanchez     prof_mmsanchez
jjsabonsolin  prof_jjsabonsolin
```

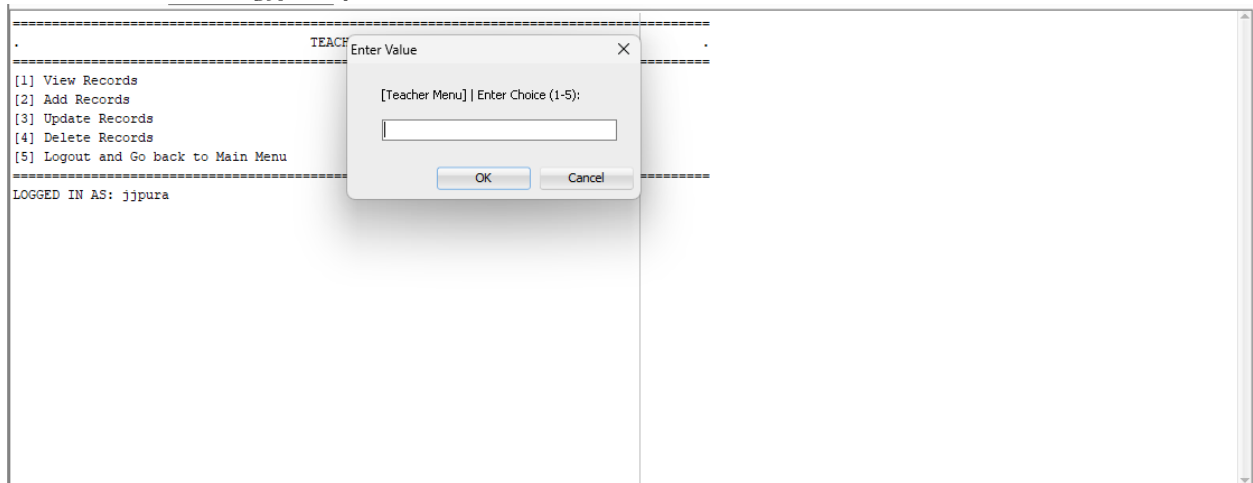
Login screens: Screenshot for student login and teacher login prompts.

```
Script Output x
Task completed in 69.496 seconds

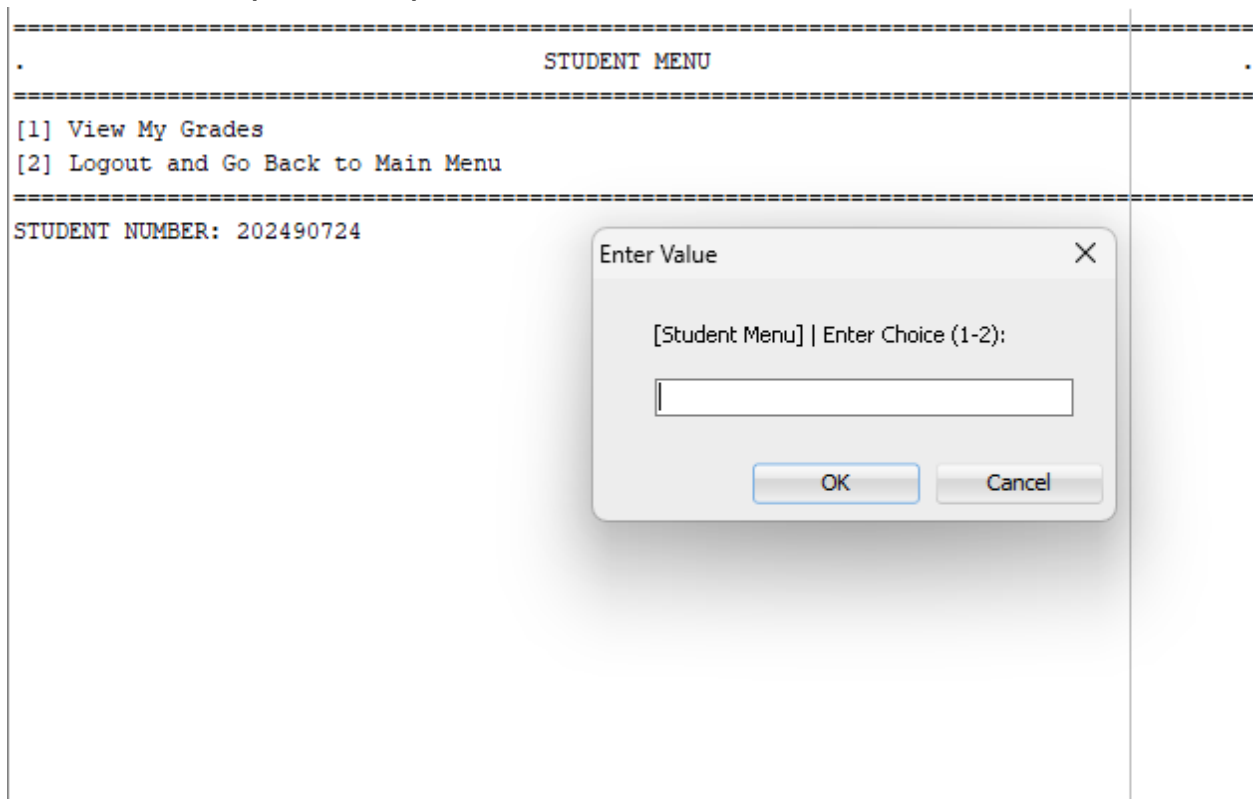
=====
.                JVS STUDENT MANAGEMENT SYSTEM                .
=====
[1] Login as Teacher
[2] Login as Student
[3] View Developer Information
[4] Exit
=====
```

Menus: Screenshot showing student menu and teacher menu.

TEACHER MENU (jjpura)



STUDENT MENU (202490724)



View grades: Screenshot showing a student's grade report.

VIEW GRADES

STUDENT NO : 202490724

NAME : HAZEL DIORNO

SECTION : TN22 | Subject 1 of 6

COURSE : CCS0101 - DESIGN THINKING

PROF : May Florence San Pablo

SA	TA	SW	REC	FP	ME	FE
50	68	87	86	96	58	57

MIDTERM	FINAL	GWA	STATUS
67	69	68	FAILED

[1] Next Subject

[2] Previous Subject

[3] Go back to Student Menu

Enter Value

What would you like to do next?

OK

Cancel

Updating records: Screenshot showing updated scores and confirmation messages.

CS0007

CS0010

CS0017

CS0021

CURRENT GRADES FOR CS0010

69	63	50	50	82	96	87
----	----	----	----	----	----	----

[Trigger] Recalculated results for Student. 202490724

Grades successfully updated.

UPDATED GRADES SUMMARY

74	78	50	70	90	89	88	78
----	----	----	----	----	----	----	----

Continue ?

Press ENTER to return to Teacher Menu...

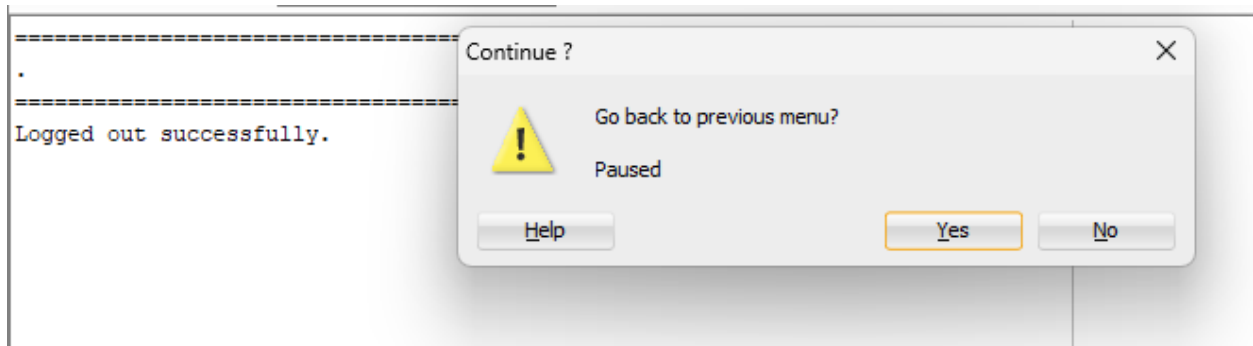
Paused

Help

Yes

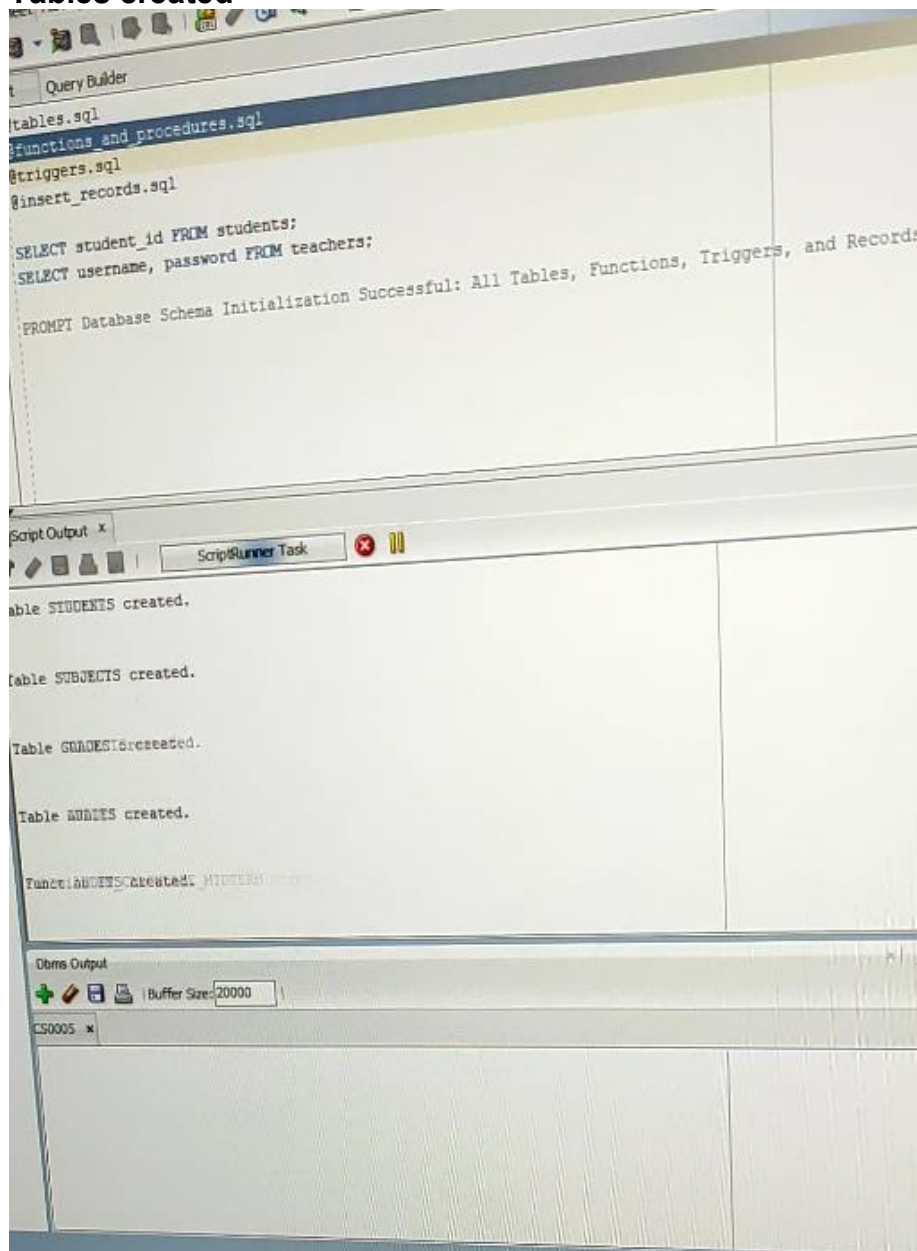
No

Logout / exit: Screenshot showing final messages after logging out or exit.

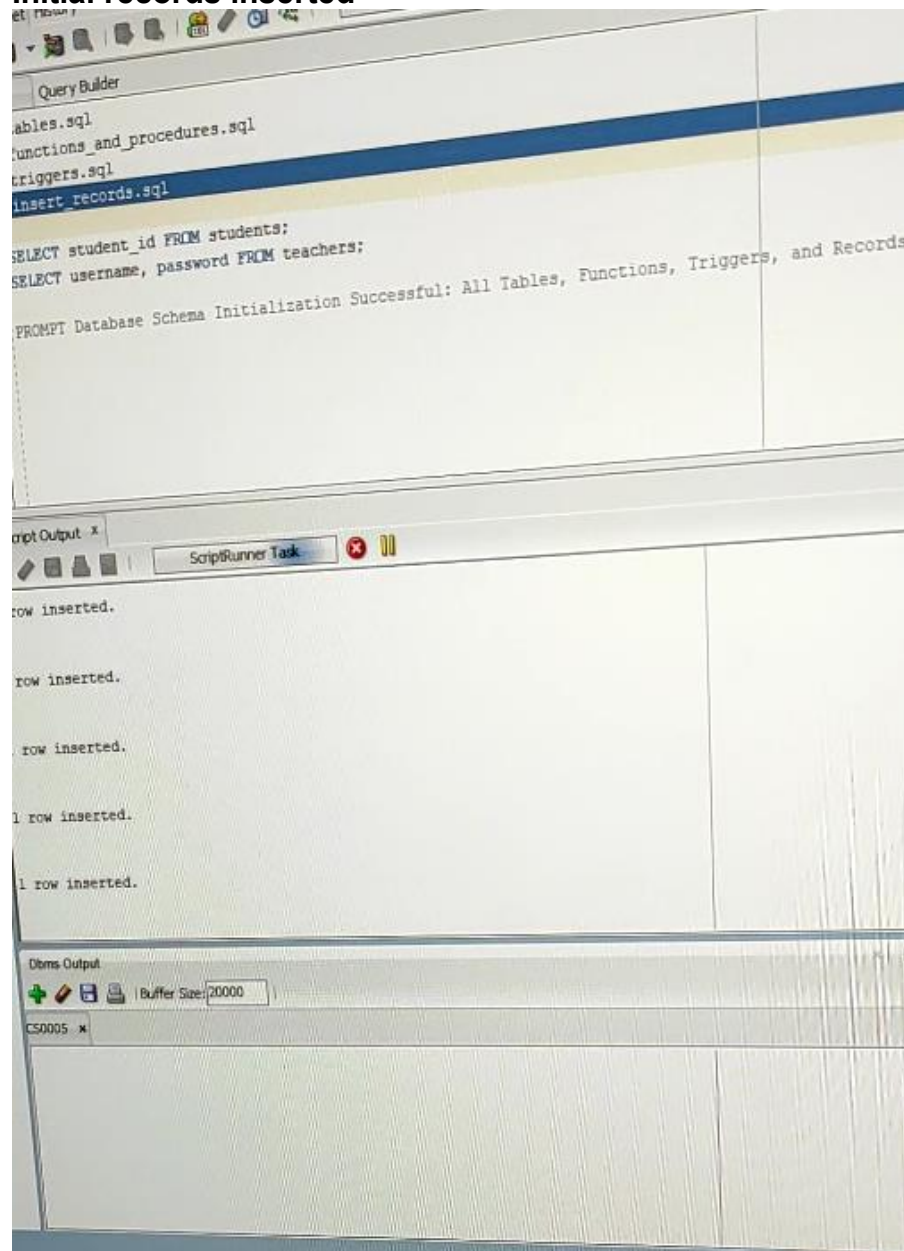


7. Output Results

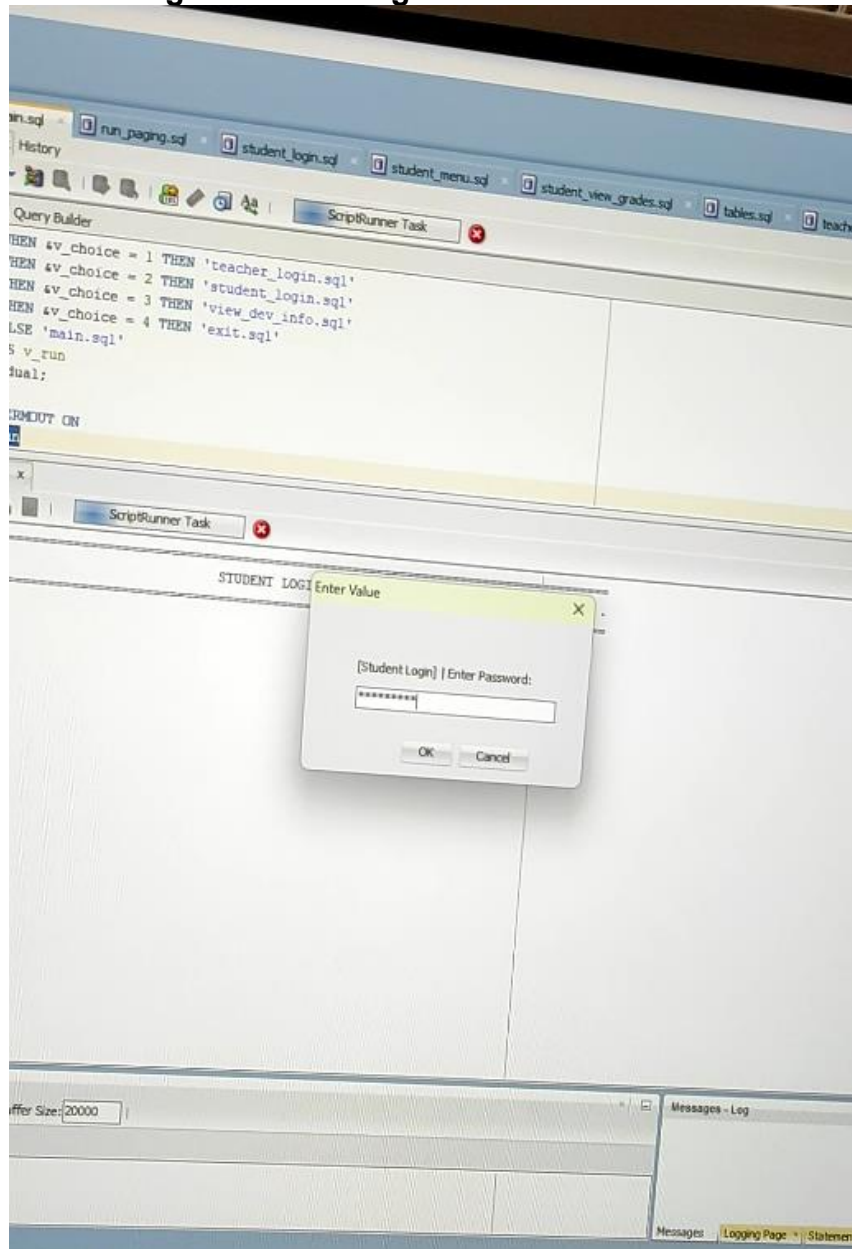
Tables created

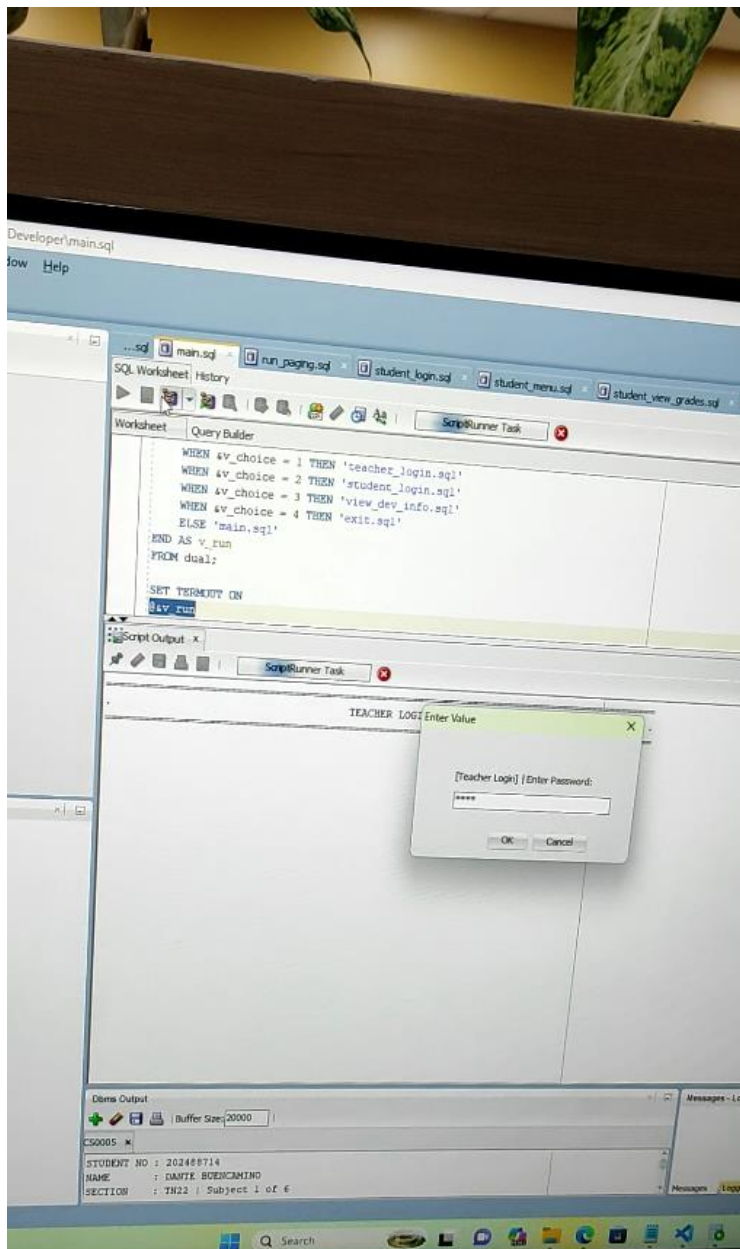


Initial records inserted



Student login / teacher login

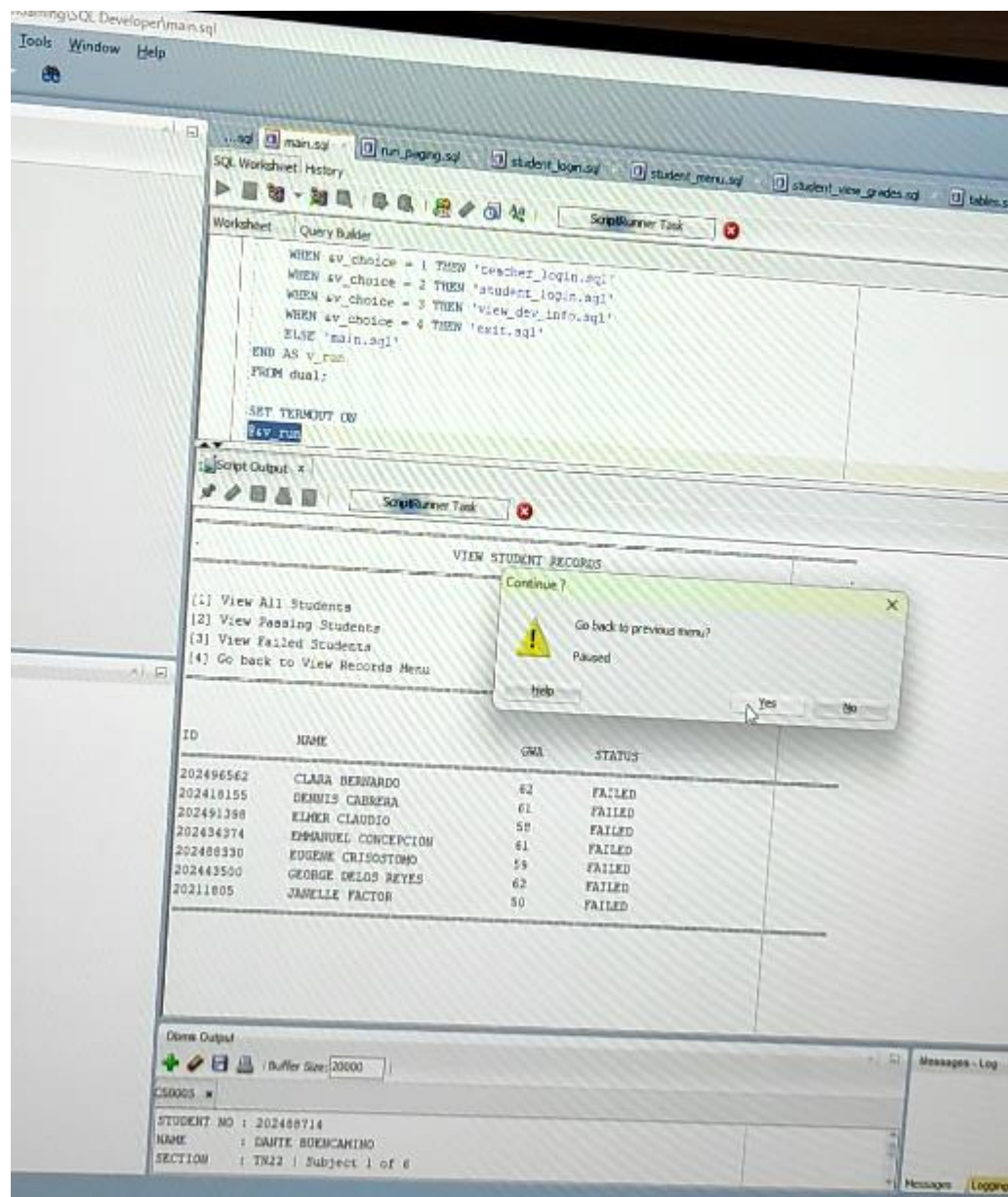




Grade calculations

C: > Users > Jan Facto > Videos > VERSION8 > functions_and_procedures.sql

```
1  -- functions_and_procedures.sql
2
3  CREATE OR REPLACE FUNCTION fn_calculate_midterm(
4      p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_me NUMBER
5  ) RETURN NUMBER IS
6      v_avg NUMBER;
7  BEGIN
8      v_avg := (p_sa + p_ta + p_sw + p_rec) / 4;
9      RETURN ROUND((0.6 * v_avg) + (0.4 * p_me));
10 END;
11 /
12
13 CREATE OR REPLACE FUNCTION fn_calculate_final(
14     p_sa NUMBER, p_ta NUMBER, p_sw NUMBER, p_rec NUMBER, p_fp NUMBER, p_me NUMBER, p_fe NUMBER
15 ) RETURN NUMBER IS
16     v_avg NUMBER;
17 BEGIN
18     v_avg := (p_sa + p_ta + p_sw + p_rec + p_fp) / 5;
19     RETURN ROUND((0.6 * v_avg) + (0.15 * p_me) + (0.25 * p_fe));
20 END;
21 /
22
23 CREATE OR REPLACE FUNCTION fn_calculate_gwa(p_mid IN NUMBER, p_fin IN NUMBER)
24 RETURN NUMBER IS
25 BEGIN
26     RETURN ROUND((p_mid + p_fin) / 2);
27 END;
28 /
29
30 CREATE OR REPLACE FUNCTION fn_get_status(p_gwa IN NUMBER)
31 RETURN VARCHAR2 IS
32 BEGIN
33     IF p_gwa >= 70 THEN
34         RETURN 'PASSED';
35     ELSE
36         RETURN 'FAILED';
37     END IF;
38 END;
39 /
```



Record updates

CS0000

CS0007

CS0010

CS0017

CS0021

=====

CURRENT GRADES FOR CS0010

=====

69

63

50

50

82

96

87

=====

[Trigger] Recalculated results for Student. 202490724

Grades successfully updated.

UPDATED GRADES SUMMARY

=====

74

78

50

70

90

89

88

78

=====

Continue ?

!

Press ENTER to return to Teacher Menu...

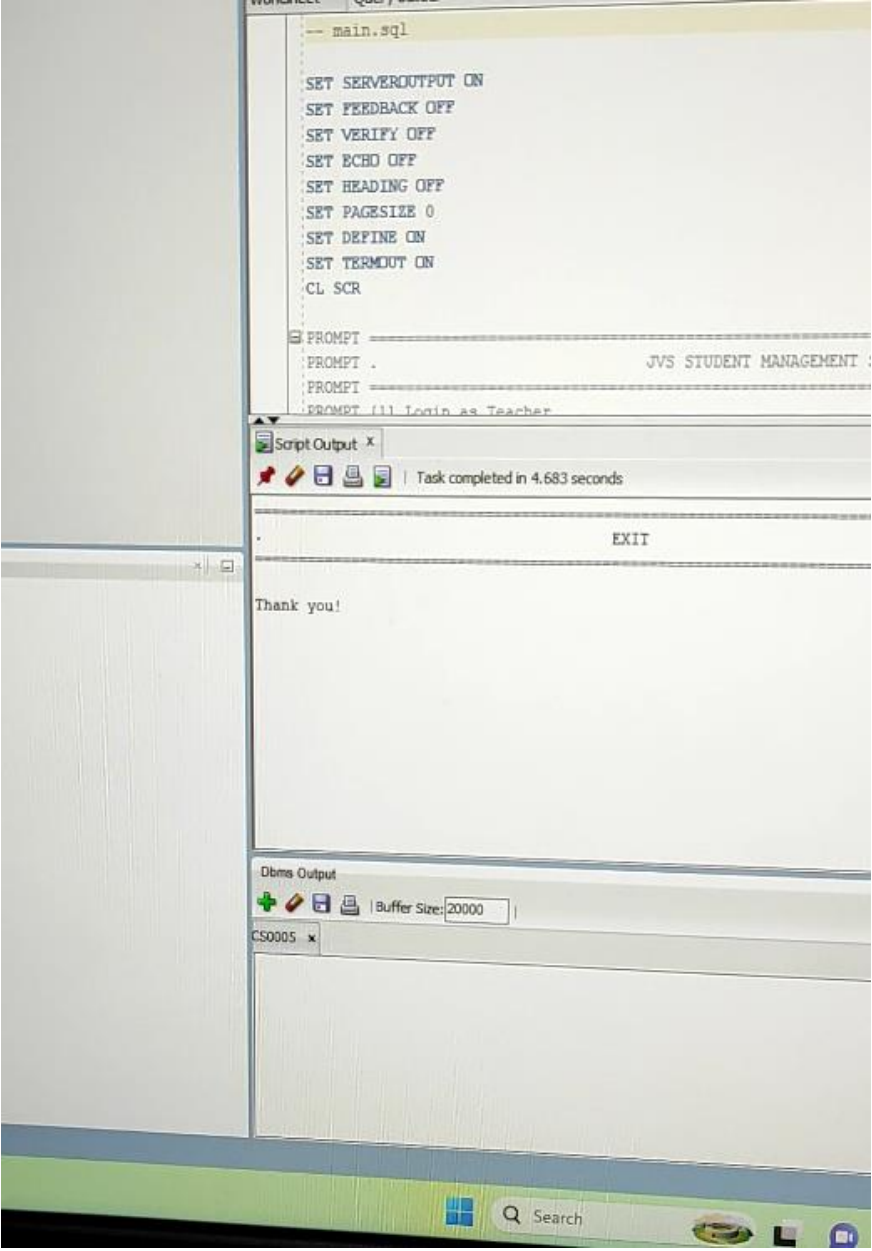
Paused

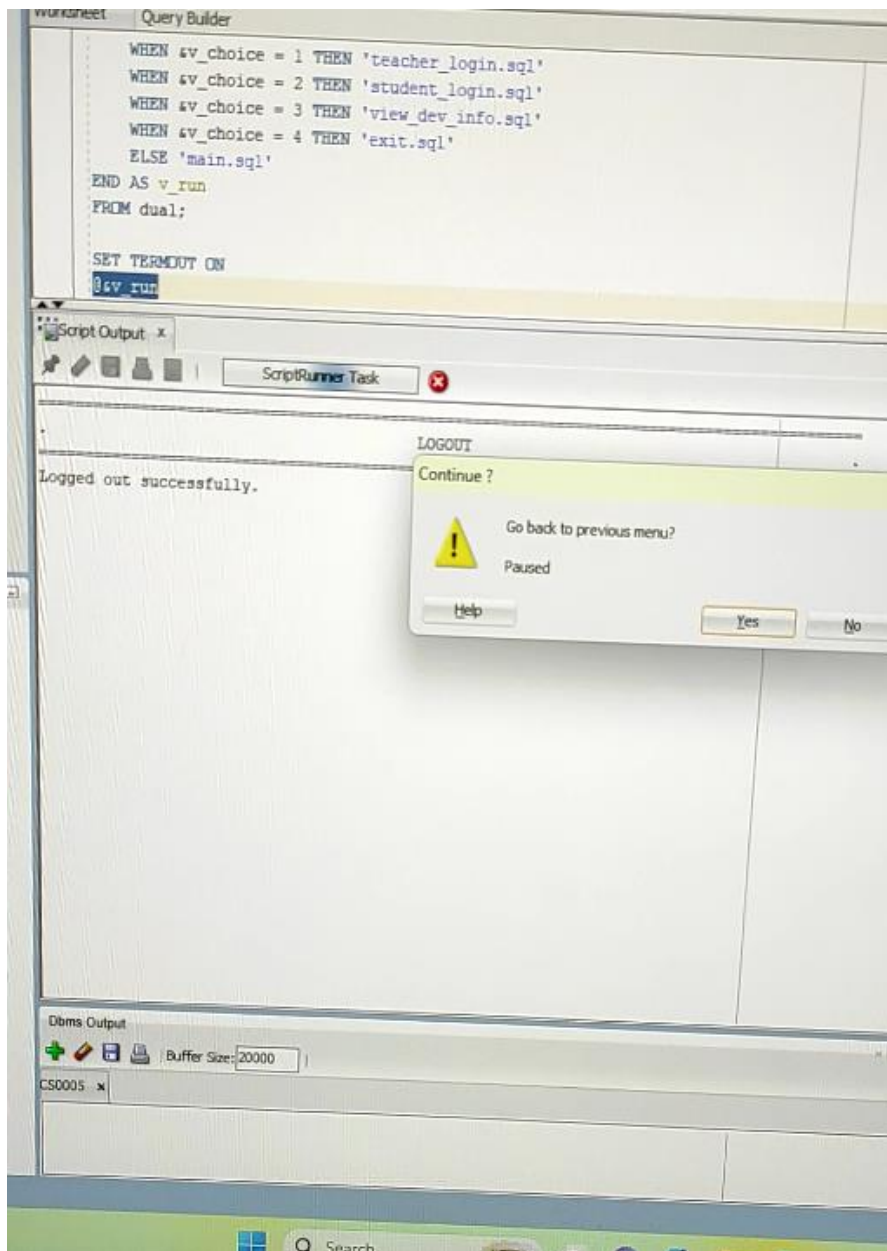
Help

Yes

No

Logout / exit messages





8. Conclusion

The system works well in handling records for teachers, students, and subjects. Students can log in to see their grades, and the calculations for midterms, finals, and overall GWA are correct. Teachers can also log in to add, update, and manage records while keeping the database accurate. Overall, the application functions as intended and achieves objectives.